

Middle Layers Know Where to Click: Coordinate Serialization Bottlenecks in GUI Agents

Anonymous submission

Abstract

Autoregressive GUI agents ground instructions by generating screen coordinates, conflating two computations that the final-layer reading leaves undistinguished: locating the target widget in screen space, and serializing that location into coordinate tokens. We show these computations peak at different transformer depths: spatial grounding is best decoded from intermediate layers and often degrades near the output, while coordinate-token predictability keeps improving in later layers. We call this mismatch the **coordinate serialization bottleneck**. To exploit it, we introduce **ZWERGE-UI**, a frozen-backbone retrofit that reads intermediate-layer posteriors through lightweight cross-attention probes ($\approx 42\text{M}$ parameters, 0.5% of an 8B backbone), fuses them with a $<200\text{K}$ -parameter head, and routes each sample between the fused point and the backbone’s own coordinate via a training-free confidence gate. Experiments across 4 GUI agents, 2 backbone families, and 5 benchmarks show that the gate yields small but consistent and statistically significant gains (paired $p < 0.001$) under the identical strict success-rate protocol, at the cost of a single forward pass.

1 Introduction

Autoregressive GUI agents ground instructions by generating screen coordinates, conflating two computations: *locating* the target widget in screen space and *serializing* that location into a coordinate vocabulary. Prior work has exploited attention maps and prefill-stage signals to aid localization (Xu et al. 2025a; Wu et al. 2025b; Lin et al. 2026), while analyses of vision transformers show that final-layer representations may lose spatial fidelity (Darcet et al. 2024; Lan et al. 2024; Chen et al. 2025). However, these lines do not ask *where within the prefill computation* spatial grounding is most accessible. At the same time, GUI-agent training studies reveal a tension between chain-of-thought reasoning and precise localization (Xu et al. 2026; Yang et al. 2026; Zhou et al. 2025; Zhang, Gao, and Xu 2025), motivating perception–reasoning decoupling (Zhang et al. 2026). We operationalize this by freezing the backbone and reading spatial beliefs directly from intermediate activations.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

We study this question with lightweight frozen-backbone readouts and reveal a consistent non-monotonic trajectory: spatial grounding improves through early and middle layers, peaks at intermediate depths, and often degrades toward the final layer. A complementary *serialization lens*—teacher-forced coordinate-token likelihood without additional parameters—shows that coordinate tokens become most predictable in later layers. This mismatch, which we call the **coordinate serialization bottleneck**, is not a generic saliency effect: an instruction-switch counterfactual shifts the mid-layer posterior from one target to another, and the bottleneck is most consequential in high-resolution layouts with small targets, where final-layer accuracy is lowest.

Motivated by this, we introduce **ZWERGE-UI**, a frozen-backbone retrofit for existing GUI agents. **ZWERGE-UI** attaches lightweight cross-attention probes to intermediate layers to recover layerwise spatial posteriors, combines them with a small cross-layer fusion head, and then applies a training-free confidence gate that decides, per sample, when to trust the intermediate-layer point over the backbone’s own autoregressive coordinate. The probes serve a dual role: they provide the analytical lens that reveals the bottleneck, and they form the grounding component used at inference. Importantly, **ZWERGE-UI** does not modify the agent backbone or its reasoning pathway.

We evaluate on four GUI agents spanning two backbone families, Qwen2.5-VL and Qwen3-VL, across five GUI grounding benchmarks, ScreenSpot-Pro, ScreenSpot-v2, OSWorld-G, MMBench-GUI, and UI-Vision, six configurations in total since we evaluate OSWorld-G on both its refusal-removed and original splits. The strongest spatial readout consistently appears before the final layer. A training-free confidence gate converts this into consistent grounding gains across all four backbones and six benchmarks (§5.2). The gains are small in magnitude but statistically significant: under a paired test on the samples where the gate and the native decoder disagree, the ensemble is net-positive in the large majority of cells and the pooled test over all samples is decisive ($p < 0.001$); no cell is significantly worse than the baseline (supplementary). We read the result as evidence that intermediate-layer signals can complement native decoding, not as a new state-of-the-art grounding system.

In summary, our contributions are:

- We identify the **coordinate serialization bottleneck**: intermediate layers encode more accurate instruction-conditioned spatial posteriors than final layers, while later layers better predict coordinate tokens.
- We introduce two complementary diagnostic lenses—a frozen-backbone *spatial lens* and a parameter-free *serialization lens*—and show a robust depth mismatch across four GUI agents.
- We present ZWERGE-UI, a lightweight frozen-backbone retrofit that recovers intermediate-layer spatial signals through cross-attention probes and a training-free, GT-free confidence gate that routes each sample between the fusion point and the native autoregressive coordinate. Across two backbone families and six benchmarks, all four retrofitted agents improve over their native baselines under the identical strict success-rate protocol, at near-zero added cost.

2 Per-Layer Spatial Probes

We diagnose *where* GUI grounding signals reside across transformer depth by attaching one lightweight probe per layer to a frozen backbone. The same probe set later forms the readout head of the retrofit (§4). Probes introduce no new spatial knowledge: the backbone is frozen, and the gradient from layer ℓ reaches only ϕ_ℓ .

Prefill-forced inference. Coordinate generation serializes a location into up to eight number tokens, but the spatial intent is already encoded before decoding begins (§3). We prefill a fixed template ending in an anchor token, `click(<|ground|><|pointer_start|>...)`, and run a single forward pass. At the anchor position the model has processed the full screenshot and instruction but no coordinate value, so there is zero label leakage.

Cross-attention probe. Let f_θ be a frozen L -layer backbone. For each probed layer $\ell \in \mathcal{L}_{\text{probe}}$, drawn from the deeper half of the network where spatial signal concentrates, we read the anchor query $\mathbf{h}_{t^*}^{(\ell)} \in \mathbb{R}^d$ at the `<|ground|>` position and visual keys $\mathbf{H}_v^{(\ell)} \in \mathbb{R}^{N_v \times d}$ over the N_v patch tokens. Each probe ϕ_ℓ computes a patch-level posterior via multi-head cross-attention with a learnable head gate α :

$$\mathbf{Q}_\ell = W_q^{(\ell)} \text{Norm}(\mathbf{h}_{t^*}^{(\ell)}), \quad \mathbf{K}_\ell = W_k^{(\ell)} \text{Norm}(\mathbf{H}_v^{(\ell)}),$$

$$p_\ell = \sigma \left(\sum_h \sigma(\alpha)_h \frac{\mathbf{Q}_{\ell,h}^\top \mathbf{K}_{\ell,h}}{\sqrt{d_h}} \right) \in \Delta^{N_v-1}, \quad (1)$$

where $\text{Norm}(\cdot)$ is the RMS pre-scale followed by LayerNorm, and $W_q^{(\ell)}, W_k^{(\ell)}$ are full-rank projections so each probe learns a geometry suited to its layer. Each probe is $\approx 4.2\text{M}$ parameters ($d=4096, n_h=8, d_h=64$); the full set totals $\approx 42\text{M}$, or 0.52% of a 7B backbone.

RMSNorm pre-scale. Hidden-state ℓ_2 norms grow with depth, geometrically misaligning layers. We pre-scale $\mathbf{h} \leftarrow \mathbf{h}/(\|\mathbf{h}\|_2/\sqrt{d})$ before each LayerNorm, bounding the input norm to $\approx \sqrt{d}$ regardless of depth and preventing bfloat16 overflow.

Supervision. Each probe is supervised with an anisotropic Gaussian centered at the bbox centroid:

$$y_i \propto \exp \left(-\frac{(x_i - c_x)^2}{2\sigma_x^2} - \frac{(y_i - c_y)^2}{2\sigma_y^2} \right), \quad \sigma_x = \eta w_b, \sigma_y = \eta h_b \quad (2)$$

which reduces over-penalization from patch-grid quantization on small targets (§3). Stage 1 minimizes $\mathcal{L}_{S1} = \frac{1}{|\mathcal{L}_{\text{probe}}|} \sum_\ell D_{\text{KL}}(\mathbf{y} \| p_\ell)$ with the backbone receiving zero gradient. Training-data construction is detailed in the supplementary material.

3 The Coordinate Serialization Bottleneck

We investigate *where* GUI grounding signals are encoded across transformer layers, progressing from the layerwise spatial profile (Finding 1), through the spatial-serialization dissociation (Finding 2) and its instruction-conditioning (Finding 3), to the resolution-dependent consequence (Finding 4).

Experimental setup. We study four GUI agents spanning two backbone families, Qwen2.5-VL (Qwen Team 2025a) and Qwen3-VL (Qwen Team 2025b), on ScreenSpot-Pro (Li et al. 2025) and ScreenSpot-v2 (Wu et al. 2025c). Spatial grounding quality is measured via a lightweight per-layer grounding head, Stage 1 of §2, which is under 1% of backbone parameters and is trained with the backbone completely frozen, so it introduces no new spatial knowledge. Coordinate serialization quality is measured via the backbone’s native language model head under teacher forcing, with no additional parameters.

3.1 Spatial Grounding Peaks at Intermediate Layers

Autoregressive GUI agents implicitly assume final-layer representations carry the richest spatial information. The blue curves in Figure 1 challenge this: spatial quality peaks at intermediate depths, then declines toward the output, a non-monotonic trajectory consistent across both backbone families.

Finding 1. *Intermediate transformer layers are consistently better spatial decoders than the final layer.*

Robustness. The inverted-U is robust to distribution and sample size: across all five benchmarks, L^* varies by ≤ 3 layers per model (std ≤ 1.1), and peak exceeds final in 20/20 cells. Bootstrap resampling (1000 resamples) places $L^\dagger$ within ≤ 1 -layer 95% CI for every model and L^* within ≤ 3 layers for three of four; $L^* < L^\dagger$ holds in all four. Full tables are in the supplementary material.

3.2 A Dissociation Between Spatial Localization and Coordinate Serialization

Finding 1 raises the question: if late layers are not spatial refiners, what do they do? We hypothesize they specialize in *coordinate serialization*—translating a latent spatial intent into the vocabulary-level token sequence. We test this with a *serialization lens*: coordinate-token log-likelihood measured via teacher forcing at each layer.

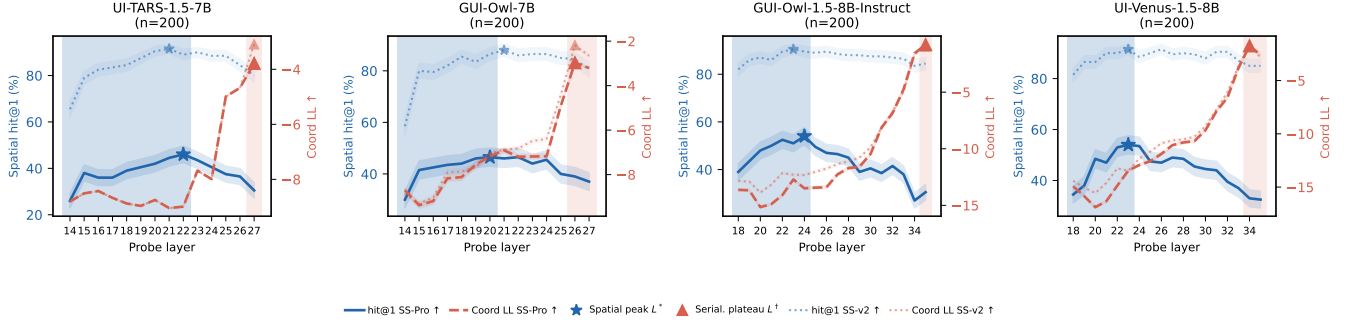


Figure 1: **Spatial localization and coordinate serialization peak at different depths.** Blue curves (left axis): spatial hit@1 ↑. Orange curves (right axis): coordinate token log-likelihood ↑. Both signals rise together, then diverge at the spatial peak L^* (blue star): spatial quality declines while serialization continues rising to $L^\dagger$ (orange triangle). The consistent lag $L^\dagger > L^*$ across all models constitutes direct evidence for the coordinate serialization bottleneck.

The orange curves in Figure 1 reveal a dissociation: both signals rise together through early layers, then diverge—spatial quality peaks and falls while coordinate log-likelihood continues climbing to a later plateau. The lag $L^\dagger > L^*$ is consistent across all four models and both backbone families, constituting direct evidence for a **coordinate serialization bottleneck**.

Finding 2. *The layer that best decodes the target region consistently precedes the layer that best predicts the coordinate token sequence. Late layers are coordinate serializers, not spatial refiners.*

Logit-lens controls. Applying the native LM head to intermediate states is a logit-lens-style probe (nostalgebraist 2020); late layers could align better with the output space for reasons unrelated to coordinates (Geva et al. 2021; Meng et al. 2022). Two controls address this; full tables are in the supplementary material.

Non-coordinate control. We repeat the teacher-forced measurement on *non-coordinate* protocol tokens (parentheses, JSON keys) from the same generations. These reach near-zero NLL at late layers for three of four models, confirming generic alignment. Crucially, coordinate *value* tokens retain a substantial residual NLL at the plateau (2.2–3.1 nats for Qwen2.5-VL; 1.0–1.7 nats for Qwen3-VL), even where format tokens are fully serialized. This residual is the coordinate-specific bottleneck component. The plateau $L^\dagger$ coincides for coordinate and protocol tokens in 3/4 models, so $L^\dagger$'s *location* is a general late-layer phenomenon while the coordinate-specific signal is the residual gap.

Tuned-lens control. Following (Belrose et al. 2023), we fit a per-layer affine map to the final-layer representation and apply the *frozen* output head. Calibration flattens the raw trend (early-layer NLL: 8.6 → 3.8 nats), showing part of the raw improvement is an alignment artifact. The plateau layer, however, is unchanged (L26 both before and after), so the *locus* is robust. Together, the residual NLL gap and

the calibration-invariant plateau indicate genuine coordinate serialization.

3.3 The Mid-Layer Posterior Is Instruction-Conditioned, Not Visual Saliency

An alternative is that mid-layer spatial signals merely reflect *visual saliency*. To test this, we design a counterfactual: for pairs sharing the same screen but targeting disjoint elements, we swap the instruction while keeping the image fixed.

Figure 2 shows the posterior shifts dramatically: switching the instruction collapses the spatial mass on the original target, concentrated at exactly the intermediate layers where grounding peaks. This rules out the saliency account.

Finding 3. *Swapping the instruction while keeping the image fixed causes a large shift in the mid-layer spatial posterior. The spatial signal is instruction-conditioned grounding, not visual saliency.*

3.4 The Bottleneck Is More Consequential on High-Resolution, Fine-Grained Layouts

Finally, we ask whether the bottleneck is more consequential where GUI grounding is hardest—high-resolution screen-shots with small, densely packed targets.

We quantify severity via the *collapse gap*: peak-layer hit@1 minus final-layer hit@1. Table 1 shows the gap is consistently larger on ScreenSpot-Pro than on ScreenSpot-v2 across every model.

Controlled within-benchmark target-size regression. SS-Pro vs SS-v2 conflates target size with resolution. To isolate size, we hold the benchmark fixed at ScreenSpot-Pro ($n=1581$) and bin by target bbox area (supplementary material). The peak-final *gap* does *not* increase monotonically as targets shrink: it rises for the two Qwen2.5-7B models but peaks at intermediate sizes for the Qwen3-8B models, floored by peak accuracy on sub-patch targets. Absolute final-layer hit@1, however, is $\sim 1.7\times$ worse on the smallest

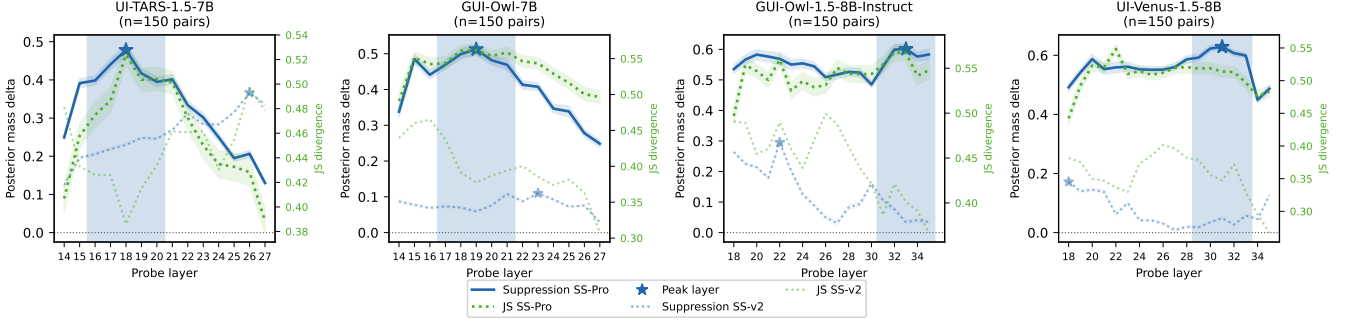


Figure 2: **Instruction-switch counterfactual: the mid-layer posterior is instruction-conditioned.** Blue curves: old-target suppression $\uparrow$. Orange curves: posterior JS divergence $\uparrow$. Both effects peak at the same intermediate layers as spatial grounding quality (cf. Fig. 1) and are substantially nonzero, ruling out visual saliency.

Model	SS-Pro			SS-v2		
	Pk _{probe}	Fin _{probe}	Gap $\downarrow$	Pk _{probe}	Fin _{probe}	Gap $\downarrow$
UI-TARS-1.5-7B	46.0	30.5	15.5	91.5	80.0	11.5
GUI-Owl-7B	46.5	37.0	9.5	88.0	80.5	7.5
GUI-Owl-1.5-8B	54.0	30.5	23.5	90.5	84.5	6.0
UI-Venus-1.5-8B	54.0	32.5	21.5	91.5	85.0	6.5

Table 1: **Collapse gap is larger on high-resolution layouts.** Pk_{probe} and Fin_{probe} are the peak-layer and final-layer *spatial-probe* hit@1 (pp) from §3.1, not the backbone’s native autoregressive accuracy reported as “Native” in Table 2; the two tables use different metrics (hit@1 here vs. overlap@1 there) and different readouts (probe vs. native decoder). SS-Pro gaps consistently exceed SS-v2 gaps.

quintile ($\sim 34\%$ vs $\sim 58\%$), so the bottleneck is most damaging in absolute terms on small targets even though the gap is not monotone in size.

Finding 4. *The collapse gap is larger on high-resolution ScreenSpot-Pro than on mid-resolution ScreenSpot-v2. Within ScreenSpot-Pro, final-layer accuracy is lowest on the smallest targets even though the collapse gap is not monotone in target size; the bottleneck is most consequential, though not largest in gap, for fine-grained layouts.*

Taken together, the four findings motivate reading the spatial posterior from intermediate layers and using cross-layer fusion to select the optimal depth per sample, as described below.

4 ZWERGE-UI

The per-layer probes of §2 are independent, and the best layer varies by sample (§3, Finding 1). We add two lightweight components on top of the frozen probes, shown in Figure 3: a cross-layer fusion head that selects the reliable depth per sample, and a training-free confidence gate

that decides when to trust the fused point over the backbone’s own decoder. Both reuse the single prefill of §2, so the retrofit adds negligible compute over the native baseline.

4.1 Cross-Layer Fusion

We draw active layers $\mathcal{L}_{\text{active}} \subset \mathcal{L}_{\text{probe}}$ from the spatial plateau identified in §3 and fuse their posteriors. A cross-layer LoRA refines each anchor query $\mathbf{h}_{t*}^{(\ell)}$ into $\tilde{\mathbf{q}}_{\ell}$; a cosine-meta scorer then assigns per-layer weights:

$$\omega_{\ell} = \text{softmax}(\alpha_{\ell} + \tau^{-1} \cos(\tilde{\mathbf{q}}_{\ell}, \mathbf{q}_{\text{meta}})), \quad p_{\text{final}} = \sum_{\ell \in \mathcal{L}_{\text{active}}} \omega_{\ell} p_{\ell}, \quad (3)$$

where α_{ℓ} is a sample-independent trainable prior on layer ℓ ’s average quality, $\mathbf{q}_{\text{meta}}$ is a global trainable meta-query serving as a soft prototype of the ideal grounding representation, and τ is a learnable temperature. Layers whose refined query aligns with $\mathbf{q}_{\text{meta}}$ receive higher weight for the current input. The LoRA contributes $\approx 200\text{K}$ parameters, two orders of magnitude below a single probe.

Stage 2 loads the Stage-1 probes; the backbone and inactive probes stay frozen, while the fusion head and the active probes remain trainable, with loss

$$\mathcal{L}_{\text{S2}} = D_{\text{KL}}(\mathbf{y} \| p_{\text{final}}) + \frac{\lambda}{|\mathcal{L}_{\text{active}}|} \sum_{\ell} D_{\text{KL}}(\mathbf{y} \| p_{\ell}), \quad \lambda = 0.2, \quad (4)$$

so the per-layer auxiliary term gives every active probe a direct gradient signal and no single layer is neglected.

4.2 Training-Free Gated Decoding

The fusion posterior p_{final} is patch-level, not a coordinate. In one prefill we read both the fused point $\mathbf{f}$, produced by a training-free posterior-to-point decoder detailed in the supplementary material, and the backbone’s native autoregressive coordinate $\mathbf{n}$, and route between them with a GT-free gate on the posterior sharpness: emit $\mathbf{f}$ when the margin $m = p_{\text{final}}^{(1)} - p_{\text{final}}^{(2)}$, the difference between the top-two single-patch probabilities, exceeds τ , otherwise emit $\mathbf{n}$. A sharp mode means the probe is confident in one location; a

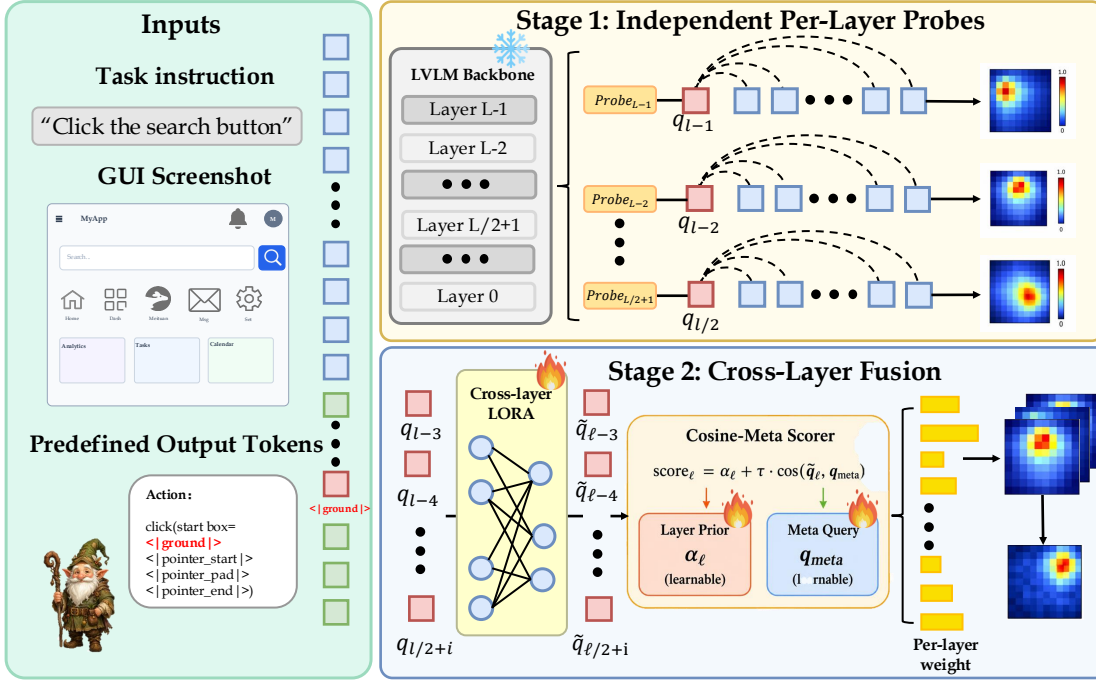


Figure 3: **Overview of ZWERGE-UI.** A frozen backbone exposes layerwise hidden states from a single prefill; independent probes decode intermediate-layer spatial posteriors, a cross-layer fusion head selects the reliable depth per sample, and a training-free confidence gate routes each sample between the fused point and the native autoregressive coordinate.

diffuse mode means the native serializer should be trusted. The default is $\tau=0.25$, while the main results use 0.20. This instantiates the paper’s thesis: the intermediate layer identifies *where* the target is, and a confidence gate decides when to trust that signal over the final-layer serializer.

Fairness and cost. The comparison against the native baseline is protocol-symmetric: both points are evaluated under the same strict point-in-box metric on the same cached run, and the gate uses no labels. The ensemble adds no autoregressive generation over the baseline. The native generate is the baseline’s own computation, and the fused point reads intermediate activations that the backbone already produces in that same forward; in principle the two share one prefill, so the full retrofit is realizable at the cost of a single forward pass.

5 Experiments

5.1 Experimental Setup

Benchmarks. We evaluate on six GUI grounding configurations: ScreenSpot-Pro (Li et al. 2025), ScreenSpot-v2 (Wu et al. 2025c), the original and refusal-removed OSWorld-G-refine splits of OSWorld-G (Xie et al. 2025), MMBench-GUI (Wang et al. 2025), and UI-Vision (Nayak et al. 2025), spanning desktop, web, and mobile.

Metrics. We report the strict single-point success rate (overlap@1); protocol and patch-mapping details are in the supplementary material.

Baselines. We retrofit four GUI agents: GUI-Owl-7B (Ye et al. 2025) and UI-TARS-1.5-7B (Qin et al. 2025), both built on the 28-layer Qwen2.5-VL, and GUI-Owl-1.5-8B (Xu et al. 2026) and UI-Venus-1.5-8B (Team et al. 2026), both built on the 36-layer Qwen3-VL. Each backbone is frozen throughout; only the probe and fusion heads are updated. The Native column is each backbone’s own autoregressive accuracy on the same task set, so the comparison isolates the retrofit.

Implementation. Stage 1 trains 14 (Qwen2.5-VL) or 18 (Qwen3-VL) CrossAttn probes per model on 200k grounding samples for one epoch; Stage 2 trains the fusion head with a subset of active probes, 10 layers for the Qwen2.5-VL backbones and 12 for the Qwen3-VL backbones, for one epoch, with the backbone and the remaining probes frozen. The deployed decoder is the margin-gated ensemble (§4.2) with $\tau=0.20$.

5.2 Main Results

Table 2 compares the confidence-gated ensemble against the native baseline under the identical strict success-rate protocol, with a single global threshold and no per-backbone tuning. Twenty of 24 cells improve and 21 of 24 are non-negative, for a macro-average gain of +0.46 pp; the three negative cells are all within 0.6 pp of zero. The gate routes to the fusion point on 5.7–32.7% of samples and otherwise defers to the native decoder; it fires most on OSWorld-G, where the native baseline has the most headroom, and least

Table 2: **Confidence-gated ensemble, strict overlap@1 (%)**. **Native**: backbone baseline. **+ZwerGe**: margin-gated ensemble (§4.2), same cached run. $\Delta = +\text{ZwerGe} - \text{Native}$. **%Fus.**: fraction routed to fusion. OSW-G = OSWorld-G (original); OSW-G-Ref = refusal-removed. Global $\tau=0.20$; 20/24 improve, 21/24 non-negative, macro-average +0.46 pp.

Backbone	Benchmark	Native ov@1	+ZwerGe ov@1	Δ	%Fus.
GUI-Owl-7B (Ye et al. 2025) Qwen2.5-VL	SS-Pro	55.22	56.10	+0.89	7.5
	SS-v2	92.84	93.08	+0.24	5.8
	OSW-G-Ref	79.61	80.39	+0.78	25.7
	OSW-G	72.55	72.94	+0.39	25.3
	MMBench-GUI	80.77	81.27	+0.50	7.5
	UI-Vision	53.87	54.42	+0.55	12.7
UI-TARS-1.5-7B (Qin et al. 2025) Qwen2.5-VL	SS-Pro	48.32	49.59	+1.27	6.8
	SS-v2	91.35	91.66	+0.31	5.7
	OSW-G-Ref	77.06	76.47	−0.59	24.5
	OSW-G	64.51	65.10	+0.59	20.8
	MMBench-GUI	77.46	77.88	+0.42	7.6
	UI-Vision	27.12	28.45	+1.33	7.6
GUI-Owl-1.5-8B (Xu et al. 2026) Qwen3-VL	SS-Pro	74.89	75.21	+0.32	10.2
	SS-v2	94.10	94.02	−0.08	9.9
	OSW-G-Ref	80.98	82.35	+1.37	32.7
	OSW-G	78.43	78.63	+0.20	30.4
	MMBench-GUI	83.58	84.17	+0.58	9.9
	UI-Vision	43.94	44.97	+1.04	14.2
UI-Venus-1.5-8B (Team et al. 2026) Qwen3-VL	SS-Pro	70.52	70.65	+0.13	7.7
	SS-v2	96.85	96.85	0.00	8.6
	OSW-G-Ref	86.08	86.27	+0.20	28.8
	OSW-G	81.76	82.35	+0.59	25.5
	MMBench-GUI	89.04	88.95	−0.08	8.8
	UI-Vision	52.18	52.33	+0.16	11.8

on ScreenSpot-v2, where native accuracy is near ceiling.

Interpreting the gains. We read these small, mostly positive gains as evidence that intermediate-layer signals can complement native decoding, not as a new state-of-the-art result. The magnitude is small by construction: the gate defers to the native decoder on the majority of samples and overrides only when its posterior is sharply peaked. On the samples where the two disagree, the ensemble is net-positive in the large majority of cells, and a pooled paired test over all samples is decisive in its favor ($p < 10^{-17}$); the gain is individually significant ($p < 0.05$) on the six highest-headroom cells, and no cell is significantly worse than the baseline (per-cell table in the supplementary).

5.3 Diagnosing the Gains: Target Geometry and Retrofit Headroom

We explain the two patterns in Table 2, the largest headroom on ScreenSpot-Pro and the smallest gains on Qwen3-VL, with two diagnostics computed from the same cached runs.

Why ScreenSpot-Pro is the hard case. Roughly three quarters of ScreenSpot-Pro targets are smaller than a single visual patch (see supplementary), so a patch-level posterior can at best identify the correct patch, not the sub-patch pixel. This is precisely why the retrofit is framed as a *gate* rather than a replacement decoder: the intermediate posterior identifies *which patch* contains the target, while sub-patch pre-

cision is left to the native serializer whenever the probe is uncertain.

Retrofit headroom predicts the gate’s behavior. Define the *retrofit headroom* $h = \max_{\ell} \text{ov}@1(p_{\ell}) - \text{ov}@1(\text{native})$. When $h > 0$, bypassing the serializer recovers real headroom; when $h \leq 0$, the native decoder has already closed that gap. A supplementary figure plots h against the ungated fusion-only delta. Qwen2.5-VL backbones, whose native decoders leave more headroom, occupy the positive-headroom, positive-gain quadrant; Qwen3-VL backbones, whose native decoders approach the intermediate-layer ceiling, sit near or below zero. This is the regime the confidence gate targets: it defers to the native decoder whenever the fusion posterior of Eq. 3 is diffuse, which is why most cells in Table 2 are non-negative.

6 Conclusion

We presented ZWERGE-UI, a frozen-backbone retrofit that exposes and exploits the coordinate serialization bottleneck in GUI agents: instruction-conditioned spatial posteriors peak at intermediate transformer layers, while later layers increasingly specialize in coordinate serialization rather than localization refinement. A training-free, GT-free confidence gate combines the fused intermediate-layer point with the native decoder at the cost of a single forward pass. Across two backbone families and six benchmarks, all four

retrofitted agents improve over their native baselines, and the improvement is statistically significant under a paired test ($p < 0.001$). The gains are modest by design: the retrofit’s purpose is to validate the bottleneck hypothesis rather than chase leaderboard scores, and the heads train on far less data than the backbone, leaving clear room for improvement as GUI agents scale.

References

- Abnar, S.; and Zuidema, W. 2020. Quantifying Attention Flow in Transformers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 4190–4197.
- Bai, J.; Bai, S.; Yang, S.; Wang, S.; Tan, S.; Wang, P.; Lin, J.; Zhou, C.; and Zhou, J. 2023. Qwen-VL: A Versatile Vision-Language Model for Understanding, Localization, Text Reading, and Beyond. *arXiv preprint arXiv:2308.12966*.
- Bai, S.; Chen, K.; Liu, X.; Wang, J.; Ge, W.; Song, S.; Dang, K.; Wang, P.; Wang, S.; Tang, J.; et al. 2025. Qwen2.5-VL Technical Report. *arXiv preprint arXiv:2502.13923*.
- Belrose, N.; Furman, Z.; Smith, L.; Halawi, D.; Ostrovsky, I.; McKinney, L.; Biderman, S.; and Steinhardt, J. 2023. Eliciting Latent Predictions from Transformers with the Tuned Lens. *arXiv preprint arXiv:2303.08112*.
- Cao, Q.; Paranjape, B.; and Hajishirzi, H. 2023. PuMer: Pruning and Merging Tokens for Efficient Vision Language Models. *arXiv preprint arXiv:2305.17530*.
- Carion, N.; Massa, F.; Synnaeve, G.; Usunier, N.; Kirillov, A.; and Zagoruyko, S. 2020. End-to-End Object Detection with Transformers. In *European Conference on Computer Vision*, 213–229.
- Chefer, H.; Gur, S.; and Wolf, L. 2021. Transformer Interpretability Beyond Attention Visualization. In *CVPR*, 782–791.
- Chen, H.; Lin, J.; Chen, X.; Fan, Y.; Jin, X.; Su, H.; Dong, J.; Fu, J.; and Shen, X. 2025. Rethinking visual layer selection in multimodal llms. *arXiv e-prints*, arXiv–2504.
- Chen, L.; Zhao, H.; Liu, T.; Bai, S.; Lin, J.; Zhou, C.; and Chang, B. 2024. An Image is Worth 1/2 Tokens After Layer 2: Plug-and-Play Inference Acceleration for Large Vision-Language Models. In *European Conference on Computer Vision*.
- Cheng, K.; Sun, Q.; Chu, Y.; Xu, F.; YanTao, L.; Zhang, J.; and Wu, Z. 2024. SeeClick: Harnessing gui grounding for advanced visual gui agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 9313–9332.
- Darcet, T.; Oquab, M.; Mairal, J.; and Bojanowski, P. 2024. Vision Transformers Need Registers. In *ICLR*.
- Dosovitskiy, A.; Beyer, L.; Kolesnikov, A.; Weissenborn, D.; Zhai, X.; Unterthiner, T.; Dehghani, M.; Minderer, M.; Heigold, G.; Gelly, S.; et al. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *ICLR*.
- Fan, Y.; Ding, L.; Kuo, C.-C.; Jiang, S.; Zhao, Y.; Guan, X.; Yang, J.; Zhang, Y.; and Wang, X. E. 2024. Read Anywhere Pointed: Layout-aware GUI Screen Reading with Tree-of-Lens Grounding. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 9503–9522. Miami, Florida, USA: Association for Computational Linguistics.
- Geva, M.; Schuster, R.; Berant, J.; and Levy, O. 2021. Transformer Feed-Forward Layers Are Key-Value Memories. In *EMNLP*.
- Gou, B.; Wang, R.; Zheng, B.; Xie, Y.; Chang, C.; Shu, Y.; Sun, H.; and Su, Y. 2025. Navigating the digital world as humans do: Universal visual grounding for gui agents. In *ICLR*.
- Gu, Z.; Zeng, Z.; Xu, Z.; Zhou, X.; Shen, S.; Liu, Y.; Zhou, B.; Meng, C.; Xia, T.; Chen, W.; et al. 2025. Ui-venus technical report: Building high-performance ui agents with rft. *arXiv preprint arXiv:2508.10833*.
- Hong, W.; Wang, W.; Lv, Q.; Xu, J.; Yu, W.; Ji, J.; Wang, Y.; Wang, Z.; Dong, Y.; Ding, M.; et al. 2024. Cogagent: A visual language model for gui agents. In *CVPR*, 14281–14290.
- Kamath, A.; Singh, M.; LeCun, Y.; Synnaeve, G.; Misra, I.; and Carion, N. 2021. MDETR: Modulated Detection for End-to-End Multi-Modal Understanding. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 1780–1790.
- Kang, S.; Kim, J.; Kim, J.; and Hwang, S. J. 2025. See What You Are Told: Visual Attention Sink in Large Multimodal Models. In *ICLR*.
- Kim, W.; Son, B.; and Kim, I. 2021. ViLT: Vision-and-Language Transformer Without Convolution or Region Supervision. In *International Conference on Machine Learning*, 5583–5594.
- Kirillov, A.; Mintun, E.; Ravi, N.; Mao, H.; Rolland, C.; Gustafson, L.; Xiao, T.; Whitehead, S.; Berg, A. C.; Lo, W.-Y.; et al. 2023. Segment Anything. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 4015–4026.
- Lan, M.; Chen, C.; Ke, Y.; Wang, X.; Feng, L.; and Zhang, W. 2024. Clearclip: Decomposing clip representations for dense vision-language inference. In *ECCV*, 143–160. Springer.
- Li, K.; Meng, Z.; Lin, H.; Luo, Z.; Tian, Y.; Ma, J.; Huang, Z.; and Chua, T.-S. 2025. Screenspot-pro: Gui grounding for professional high-resolution computer use. In *ACM MM*, 8778–8786.
- Li, L. H.; Zhang, P.; Zhang, H.; Yang, J.; Li, C.; Zhong, Y.; Wang, L.; Yuan, L.; Zhang, L.; Hwang, J.-N.; et al. 2022. Grounded Language-Image Pre-training. In *CVPR*, 10965–10975.
- Li, W.; Yuan, Y.; Liu, J.; Tang, D.; Wang, S.; Qin, J.; Zhu, J.; and Zhang, L. 2024. TokenPacker: Efficient Visual Projector for Multimodal LLM. *arXiv preprint arXiv:2407.02392*.
- Lin, J.; Shen, F.; Li, J.; Nie, P.; Yu, F.; Li, M.; and Li, H. 2026. What Happens Before Decoding? Prefill Determines GUI Grounding in VLMs. *arXiv preprint arXiv:2605.12549*.

- Lin, K. Q.; Li, L.; Gao, D.; Yang, Z.; Wu, S.; Bai, Z.; Lei, S. W.; Wang, L.; and Shou, M. Z. 2025. Showui: One vision-language-action model for gui visual agent. In *CVPR*, 19498–19508.
- Liu, S.; Zeng, Z.; Ren, T.; Li, F.; Zhang, H.; Yang, J.; Jiang, Q.; Li, C.; Yang, J.; Su, H.; et al. 2024. Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection. In *European Conference on Computer Vision*.
- Liu, Y.; Li, P.; Xie, C.; Hu, X.; Han, X.; Zhang, S.; Yang, H.; and Wu, F. 2025. Infigui-r1: Advancing multimodal gui agents from reactive actors to deliberative reasoners. *arXiv preprint arXiv:2504.14239*.
- Liu, Y.; Liu, Z.; Zhu, S.; Li, P.; Xie, C.; Wang, J.; Hu, X.; Han, X.; Yuan, J.; Wang, X.; et al. 2026. Infigui-g1: Advancing gui grounding with adaptive exploration policy optimization. In *AAAI*, 38, 32267–32275.
- Lu, Y.; Yang, J.; Shen, Y.; and Awadallah, A. 2024. Omniparser for pure vision based gui agent. *arXiv preprint arXiv:2408.00203*.
- Lu, Z.; Chai, Y.; Guo, Y.; Yin, X.; Liu, L.; Wang, H.; Xiao, H.; Ren, S.; Xiong, G.; and Li, H. 2025. UI-R1: Enhancing Efficient Action Prediction of GUI Agents by Reinforcement Learning. *arXiv preprint arXiv:2503.21620*.
- Luo, R.; Wang, L.; He, W.; Chen, L.; Li, J.; and Xia, X. 2025a. Gui-r1: A generalist r1-style vision-language action model for gui agents. *arXiv preprint arXiv:2504.10458*.
- Luo, T.; Logeswaran, L.; Johnson, J.; and Lee, H. 2025b. Visual Test-time Scaling for GUI Agent Grounding. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*.
- Meng, K.; Bau, D.; Andonian, A.; and Belinkov, Y. 2022. Locating and Editing Factual Associations in GPT. *NeurIPS*.
- Minderer, M.; Gritsenko, A.; Stone, A.; Neumann, M.; Weissenborn, D.; Dosovitskiy, A.; Mahendran, A.; Arnab, A.; Dehghani, M.; Shen, Z.; et al. 2022. Simple Open-Vocabulary Object Detection with Vision Transformers. In *European Conference on Computer Vision*.
- Nayak, S.; Jian, X.; Lin, K. Q.; Rodriguez, J. A.; Kalsi, M.; Awal, R.; Chapados, N.; Özsu, M. T.; Agrawal, A.; Vazquez, D.; et al. 2025. Ui-vision: A desktop-centric gui benchmark for visual perception and interaction. In *ICML*.
- nostalgebraist. 2020. Interpreting GPT: the logit lens. LessWrong.
- Ouyang, M.; Lin, K. Q.; Shou, M. Z.; and Ng, H. T. 2026. FocusUI: Efficient UI Grounding via Position-Preserving Visual Token Selection. *arXiv preprint arXiv:2601.03928*.
- Qin, Y.; Ye, Y.; Fang, J.; Wang, H.; Liang, S.; Tian, S.; Zhang, J.; Li, J.; Li, Y.; Huang, S.; et al. 2025. UI-TARS: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326*.
- Qwen Team. 2025a. Qwen2.5-VL technical report. *arXiv preprint arXiv:2502.13923*.
- Qwen Team. 2025b. Qwen3-VL Technical Report. *arXiv preprint arXiv:2511.21631*.
- Radford, A.; Kim, J. W.; Hallacy, C.; Ramesh, A.; Goh, G.; Agarwal, S.; Sastry, G.; Askell, A.; Mishkin, P.; Clark, J.; et al. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *International Conference on Machine Learning*, 8748–8763.
- Sun, L.; Chen, X.; Chen, L.; Dai, T.; Zhu, Z.; and Yu, K. 2022. META-GUI: Towards Multi-modal Conversational Agents on Mobile GUI. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 6699–6712. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics.
- Team, V.; Gao, C.; Gu, Z.; Liu, Y.; Qiu, X.; Shen, S.; Wen, Y.; Xia, T.; Xu, Z.; Zeng, Z.; et al. 2026. Ui-venus-1.5 technical report. *arXiv preprint arXiv:2602.09082*.
- Wang, X.; Wu, Z.; Xie, J.; Ding, Z.; Yang, B.; Li, Z.; Liu, Z.; Li, Q.; Dong, X.; Chen, Z.; et al. 2025. Mmbench-gui: Hierarchical multi-platform evaluation framework for gui agents. *arXiv preprint arXiv:2507.19478*.
- Wu, H.; Chen, H.; Cai, Y.; Liu, C.; Ye, Q.; Yang, M.-H.; and Wang, Y. 2025a. DiMo-GUI: Advancing Test-time Scaling in GUI Grounding via Modality-Aware Visual Reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 26246–26256.
- Wu, Q.; Cheng, K.; Yang, R.; Zhang, C.; Yang, J.; Jiang, H.; Mu, J.; Peng, B.; Qiao, B.; Tan, R.; et al. 2025b. Gui-actor: Coordinate-free visual grounding for gui agents. In *NeurIPS*.
- Wu, Z.; Wu, Z.; Xu, F.; Wang, Y.; Sun, Q.; Jia, C.; Cheng, K.; Ding, Z.; Chen, L.; Liang, P. P.; et al. 2025c. Os-atlas: A foundation action model for generalist gui agents. In *ICLR*.
- Xiao, G.; Tian, Y.; Chen, B.; Han, S.; and Lewis, M. 2024. Efficient Streaming Language Models with Attention Sinks. In *ICLR*.
- Xie, T.; Deng, J.; Li, X.; Yang, J.; Wu, H.; Chen, J.; Hu, W.; Wang, X.; Xu, Y.; Wang, Z.; et al. 2025. Scaling computer-use grounding via user interface decomposition and synthesis. In *NeurIPS*.
- Xu, H.; Zhang, X.; Liu, H.; Wang, J.; Zhu, Z.; Zhou, S.; Hu, X.; Gao, F.; Cao, J.; Wang, Z.; et al. 2026. Mobile-agent-v3. 5: Multi-platform fundamental gui agents. *arXiv preprint arXiv:2602.16855*.
- Xu, H.-M.; Chen, Q.; Wang, L.; and Liu, L. 2025a. Attention-driven gui grounding: Leveraging pretrained multimodal large language models without fine-tuning. In *AAAI*, 8851–8859.
- Xu, Y.; Wang, Z.; Wang, J.; Lu, D.; Xie, T.; Saha, A.; Sahoo, D.; Yu, T.; and Xiong, C. 2025b. Aguis: Unified pure vision agents for autonomous gui interaction. *ICML*.
- Yang, R.; Wu, Q.; Wang, Z.; Chen, H.; Yang, K.; Cheng, H.; Yao, H.; Peng, B.; Zhang, H.; Gao, J.; et al. 2026. Gui-libra: Training native gui agents to reason and act with action-aware supervision and partially verifiable rl. *arXiv preprint arXiv:2602.22190*.
- Ye, J.; Zhang, X.; Xu, H.; Liu, H.; Wang, J.; Zhu, Z.; Zheng, Z.; Gao, F.; Cao, J.; Lu, Z.; et al. 2025. Mobile-Agent-V3: Fundamental Agents for GUI Automation. *arXiv preprint arXiv:2508.15144*.

You, K.; Zhang, H.; Schoop, E.; Weers, F.; Swearngin, A.; Nichols, J.; Yang, Y.; and Gan, Z. 2024. Ferret-UI: Grounded Mobile UI Understanding with Multimodal LLMs. In *European Conference on Computer Vision*.

Zhang, L.; Gao, L.; and Xu, M. 2025. Does chain-of-thought reasoning help mobile gui agent? an empirical study. *arXiv preprint arXiv:2503.16788*.

Zhang, L.; Xiao, Y.; Lu, X.; Cao, J.; Zhao, Y.; Zhou, J.; An, L.; Feng, Z.; Sha, W.; Shi, Y.; et al. 2026. OmegaUse: Building a General-Purpose GUI Agent for Autonomous Task Execution. *arXiv preprint arXiv:2601.20380*.

Zhou, Y.; Dai, S.; Wang, S.; Zhou, K.; Jia, Q.; and Xu, J. 2025. Gui-g1: Understanding r1-zero-like training for visual grounding in gui agents. In *NeurIPS*.

Supplementary Material

A Related Work

GUI grounding. GUI grounding maps a natural-language instruction to an executable screen location, and is harder than general visual grounding because of dense text, small icons, regular layouts, and high-resolution screenshots (Li et al. 2025). Early screenshot-based agents such as META-GUI (Sun et al. 2022), CogAgent (Hong et al. 2024), SeeClick (Cheng et al. 2024), Ferret-UI (You et al. 2024), and OmniParser (Lu et al. 2024) showed that visual grounding can replace reliance on HTML, DOM, or accessibility trees, and subsequent work scaled model capacity and action traces, including UGround (Gou et al. 2025), OS-Atlas (Wu et al. 2025c), ShowUI (Lin et al. 2025), Aguis (Xu et al. 2025b), UI-TARS (Qin et al. 2025), GUI-Owl (Ye et al. 2025; Xu et al. 2026), and UI-Venus (Gu et al. 2025; Team et al. 2026), the four backbones we retrofit. Reinforcement-learning variants such as UI-R1 (Lu et al. 2025), GUI-R1 (Luo et al. 2025a), InfiGUI-R1 (Liu et al. 2025), GUI-G1 (Zhou et al. 2025), and InfiGUI-G1 (Liu et al. 2026) further refine coordinate prediction with grounding-specific rewards, but all read out the final autoregressive layer, the assumption our analysis (§3) revisits.

Coordinate-free and region-centric grounding. Because coordinate regression is numerically unstable and its point-level supervision is ambiguous, several methods instead read a region-level posterior. Tree-of-Lens (Fan et al. 2024) builds hierarchical layout structures from user-indicated points; TAG (Xu et al. 2025a), RegionFocus (Luo et al. 2025b), DiMo-GUI (Wu et al. 2025a), and FocusUI (Ouyang et al. 2026) mine intrinsic attention, rescale the viewport, or select position-preserving visual tokens at test time. GUI-Actor (Wu et al. 2025b) is closest in spirit: a coordinate-free, frozen-backbone head reported on ScreenSpot-Pro with Qwen2/Qwen2.5 backbones. It attaches a single $\sim 25\text{M}$ -parameter action head to the *final* layer with no cross-layer fusion; ZWERGE-UI instead distributes $\approx 51\text{--}75\text{M}$ parameters across independent probes on *multiple intermediate* layers plus a $\approx 200\text{K}$ -parameter fusion head, motivated by our finding that the most spatially informative layer is backbone-dependent and typically precedes the final layer (§3.1), so a single last-layer head cannot recover it. We view ZWERGE-UI as a multi-layer, depth-aware generalization of this coordinate-free family. Re-Prefill (Lin et al. 2026) is concurrent and complementary: it studies the *stage-level* asymmetry between prefill and decoding, showing the prefill stage already fixes candidate elements, and re-prefills with attention guidance to refine them. We instead study the *layer-level* asymmetry *within* a single prefill, showing that spatial localization and coordinate serialization peak at different depths; the two views address different granularities of the same underlying observation that GUI grounding is not solely a final-layer, autoregressive phenomenon.

Where does grounding live inside the network? Our diagnostic lenses build on a broader literature examining

what individual layers and tokens encode. Logit-lens probing decodes intermediate hidden states with the frozen output head (nostalgebraist 2020), and tuned-lens learns a calibrated per-layer affine map to correct for representational drift before decoding (Belrose et al. 2023), the technique we adapt as a control for our serialization lens (§3.2). Layer-wise probing has also been used to localize factual and semantic content in language models (Geva et al. 2021; Meng et al. 2022), and analyses of vision transformers show that final-layer tokens can lose spatial fidelity while a small set of high-norm “register” tokens absorb global computation instead of encoding local content (Darcet et al. 2024; Lan et al. 2024; Chen et al. 2025). This echoes attention-sink behavior documented in LLMs (Xiao et al. 2024) and in large multimodal models (Kang et al. 2025), where high-attention or high-norm tokens serve internal bookkeeping rather than task-relevant regions. Attention-based explanation methods such as Attention Rollout (Abnar and Zuidema 2020) and relevance propagation (Chefer, Gur, and Wolf 2021) likewise assume the final layer is the right place to look; our norm census (supplementary material) shows the same depth-dependent magnitude growth these works report, but for GUI spatial posteriors rather than classification tokens, and ties it directly to a measurable grounding-accuracy collapse rather than a purely representational curiosity. Together, this literature motivates treating layer depth as a first-class design axis rather than reading out only the last layer, which is the premise ZWERGE-UI operationalizes for GUI grounding.

Efficient multimodal grounding backbones. GUI screenshots are high-resolution, so the visual token budget is a practical constraint on any read-out design. Foundational vision-language architectures (Dosovitskiy et al. 2021; Radford et al. 2021; Bai et al. 2023, 2025; Qwen Team 2025b) and open-vocabulary detectors (Carion et al. 2020; Kamath et al. 2021; Li et al. 2022; Liu et al. 2024; Minderer et al. 2022; Kirillov et al. 2023) establish the patch-token representations our probes read from, while token-compression methods such as ViLT (Kim, Son, and Kim 2021), PuMer (Cao, Paranjape, and Hajishirzi 2023), FastV (Chen et al. 2024), and TokenPacker (Li et al. 2024) reduce the same visual redundancy that our patch-level posterior operates over. Our probes are complementary to this line: rather than compressing tokens before the backbone processes them, we read an existing intermediate representation after a single frozen forward pass, so the retrofit composes with any of these efficiency techniques without retraining the backbone.

B Robustness, Controls, and Statistical Stability

This appendix consolidates additional controls and uncertainty quantification, all computed on the A8 (checkpoint-2800) probes used in the main text. All scripts and raw JSON are released with the submission.

OOD layout generalization. Table 3 reports the spatial-peak layer L^* for each (model, benchmark) cell. L^* varies

by at most 3 layers within each model across the five benchmarks ($\text{std} \leq 1.1$), including the distribution-shifted UI-Vision and MMBench-GUI. The inverted-U (peak $>$ final) holds in 20/20 cells.

Bootstrap confidence intervals and seed stability. Tables 4–5 report $1000\times$ bootstrap resamples of the $n=200$ spatial-lens set (Exp. 3) and the $n=150$ instruction-switch pairs (Exp. 4). The serialization plateau $L^\dagger$ is stable within a ≤ 1 -layer 95% CI for every model; L^* within ≤ 3 layers for three of four (≤ 7 for GUI-Owl-7B); the lag $L^\dagger - L^* > 0$ holds in 4/4 models. Absolute hit@1 carries ± 2 –4 pp 95% CI, matching the few-pp discrepancy between the $n=200$ lens curves and the full $n=1581$ per-layer summaries.

Inference-time fusion ablation. Table 6 compares four read-out strategies on the same A8 CrossAttn probes: final-layer-only, oracle best-single layer (upper bound, not deployable), static uniform averaging of active-layer predictions, and the learned CosMeta fusion. Learned fusion improves over final-only by 7–12 pp overlap@1 on SS-Pro, tracks the oracle within 2 pp (and exceeds it on one model), and beats static averaging, quantitatively motivating both intermediate-layer readout and learned per-sample depth selection.

Controlled within-benchmark target-size regression. Figure 4 and Table 7 hold the benchmark fixed (ScreenSpot-Pro, $n=1581$) and bin by target bbox area. The peak–final gap does *not* increase monotonically as targets shrink for all models (it rises monotonically with size for the two Qwen2.5-7B models, but peaks at intermediate sizes and falls back for the two Qwen3-8B models; averaged over all four models it is floored by peak accuracy rather than target size); the absolute final-layer hit@1 (mean over models) is $\sim 1.7\times$ worse (range 1.5 – $1.9\times$) on the smallest quintile than on the largest, so the bottleneck is most damaging in absolute terms on small targets.

	SS-Pro	SS-v2	OSWorld-G	UI-Vision	MMBench-GUI	mean	std	range↓
UI-TARS-1.5-7B	20	21	21	23	20	21.0	1.10	3
GUI-Owl-7B	19	21	20	20	20	20.0	0.63	2
GUI-Owl-1.5-8B-Instruct	24	23	23	22	25	23.4	1.02	3
UI-Venus-1.5-8B	23	25	25	24	24	24.2	0.75	2

Table 3: **Spatial-peak layer L^* is stable across UI-layout distributions.** Each cell is the hit@1-peak layer on that benchmark (A8 ckpt-2800). Range is max – min across the five benchmarks.

Model	L^* [95% CI]	$L^\dagger$ [CI]	Lag	Peak hit@1 [CI]	Gap [CI]	n
UI-TARS-1.5-7B	22 [21, 23]	27 [27, 27]	+5	46.2 [39.5, 53.5]	15.6 [10.0, 22.0]	200
GUI-Owl-7B	21 [17, 24]	26 [26, 26]	+5	47.8 [41.5, 55.0]	10.7 [5.5, 16.5]	200
GUI-Owl-1.5-8B-Instruct	23 [21, 24]	35 [35, 35]	+12	54.6 [48.5, 61.0]	24.1 [18.0, 30.5]	200
UI-Venus-1.5-8B	23 [22, 24]	34 [34, 34]	+11	55.0 [48.5, 61.5]	22.3 [15.5, 28.5]	200

Table 4: **Bootstrap 95% CIs (Exp. 3, $n=200$).** $L^\dagger$ (serialization plateau) is stable within one layer; the bottleneck ordering $L^* < L^\dagger$ is robust.

Model	Peak layer [CI]	Supp _{max} [CI]	JS _{max} [CI]	n
UI-TARS-1.5-7B	18 [18, 18]	0.478 [0.447, 0.509]	0.524 [0.508, 0.542]	150
GUI-Owl-7B	19 [19, 19]	0.512 [0.482, 0.540]	0.562 [0.546, 0.578]	150
GUI-Owl-1.5-8B-Instruct	33 [32, 33]	0.601 [0.572, 0.633]	0.568 [0.550, 0.584]	150
UI-Venus-1.5-8B	31 [31, 31]	0.629 [0.611, 0.646]	0.514 [0.498, 0.529]	150

Table 5: **Bootstrap 95% CIs (Exp. 4 instruction-switch, $n=150$).** For the two Qwen3-8B models the suppression peak lands in late layers (L31–33), not the spatial-peak band (L23–25); we report this discrepancy honestly rather than selecting a more favorable window.

Model	SS-Pro				SS-v2			
	Final	Best*	Static	Learned	Final	Best*	Static	Learned
UI-TARS-1.5-7B	45.3	54.0	42.0	52.5	88.2	91.2	87.3	90.6
GUI-Owl-7B	51.7	59.5	47.1	60.0	87.4	91.7	86.0	90.6
GUI-Owl-1.5-8B-Instruct	57.2	70.8	58.1	68.8	89.7	93.1	90.0	92.6
UI-Venus-1.5-8B	56.9	66.9	56.4	66.3	92.5	94.4	92.8	94.0

Table 6: **Inference-time fusion ablation (overlap@1, A8 ckpt-2800).** * = oracle best single layer (per-benchmark, not deployable). On SS-Pro, learned CosMeta fusion is within 2 pp of the oracle for three of four models and exceeds it for GUI-Owl-7B (+0.5); it beats static averaging on all 8/8 (model, benchmark) cells and dominates final-layer-only throughout.

Model	Q1 (smallest)	Q2	Q3	Q4	Q5 (largest)
UI-TARS-1.5-7B	10.4	11.4	15.8	17.4	22.4
GUI-Owl-7B	6.0	8.2	15.2	15.5	20.2
GUI-Owl-1.5-8B-Instruct	21.5	24.7	25.3	25.3	17.7
UI-Venus-1.5-8B	14.2	20.9	27.2	15.8	13.9

Table 7: **Within-SS-Pro collapse gap (pp) by target-area quintile (hit@1).** The gap is not monotone-decreasing in target size; it is floored by peak-layer accuracy on the hardest (sub-patch) targets.

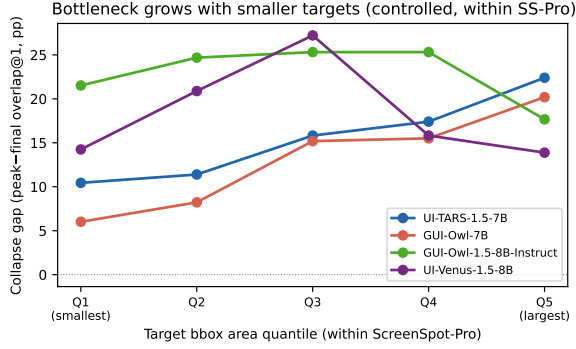


Figure 4: **Collapse gap vs target-area quintile within ScreenSpot-Pro (hit@1).** The gap does not grow as targets shrink; the absolute final-layer accuracy (not the gap) is the cleaner severity measure and is worst on the smallest targets.

Pre-scale visual-token norm census (cf. Vision Transformers Need Registers (Darcet et al. 2024)). Figure 5 measures the ℓ_2 -norm of the visual-patch hidden states *before* the RMSNorm pre-scale, i.e. the magnitude information the pre-scale discards, for all four models. The mean norm grows with depth in *both* backbone families: $\sim 3.4\times$ from layer 14 to its within-probe-range peak at layer 26 in the two Qwen2.5-VL 7B models (GUI-Owl-7B: 112 $\rightarrow$ 382; UI-TARS-1.5-7B: 110 $\rightarrow$ 371) and $\sim 7.1\text{--}7.8\times$ from layer 18 to the final probed layer 35 in the two Qwen3-VL 8B models (GUI-Owl-1.5-8B: 91 $\rightarrow$ 712; UI-Venus-1.5-8B: 88 $\rightarrow$ 630). Intriguingly, for the Qwen2.5 models the norm itself peaks at L26 and drops back at the final layer L27 (382 $\rightarrow$ 233 and 371 $\rightarrow$ 241 respectively), an echo, at the level of raw activation magnitude, of the same inverted-U depth profile we observe for spatial grounding quality (Finding 1), suggestive of a broader representational transition around the last one to two layers rather than a norm-specific artifact. This directly confirms the depth-growth claim of §2 (the deeper network grows more); a small ($\leq 0.7\%$) but heavy-tailed set of outlier patches carries very high norm, consistent with the attention-sink / register-token phenomenon, information the pre-scale normalizes away but which a magnitude-aware variant could exploit.

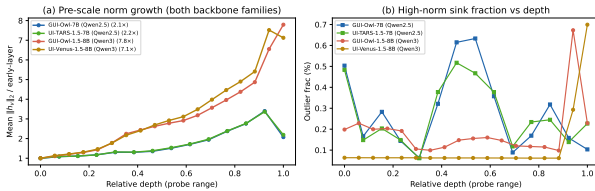


Figure 5: **Pre-scale visual-patch norm census (all four models across both backbone families, $n=100$ each).** (a) mean $\|h_v\|_2$ grows $\sim 3.4\times$ (Qwen2.5, 2 models) and $\sim 7.1\text{--}7.8\times$ (Qwen3, 2 models) with depth; (b) high-norm sink fraction stays small ($\leq 0.7\%$) but present in all four.

Tuned-lens control for the serialization lens. The raw serialization lens applies the frozen norm+LM-head to each layer’s hidden state; late layers align better with the output space, which can lower their NLL for reasons unrelated to coordinates. Following the tuned-lens methodology (Belrose et al. 2023), we learn per layer a lightweight affine $\hat{h}_\ell = A_\ell h_\ell + b_\ell$ mapping the intermediate state to the final-layer representation (ridge regression, $\lambda=10$, 80/20 train/held-out split, $N=1924$ coordinate tokens from $n=200$ ScreenSpot-Pro samples), then apply the *frozen* norm+LM-head. Table 8 reports raw vs. tuned NLL per layer. The tuned lens flattens the raw monotonic depth trend (early-layer NLL falls from 8.6 to 3.8 after calibration, part of the raw decrease is indeed an alignment artifact), yet the plateau position is unchanged: $L_{\text{raw}}^\dagger = L_{\text{tuned}}^\dagger = \text{L26}$. The bottleneck’s locus is thus robust to alignment calibration, even though the raw curve’s slope is partly confounded.

Random-instruction control. A training-free control: we run the frozen A8 probe on the same ScreenSpot-Pro images with the *real* instruction replaced by an unrelated sample’s instruction (shuffled across the batch). If the probe merely learned a position prior (“always click the most salient icon”), randomizing the instruction would not hurt. It does: hit@1 and target-region probability mass both collapse (Table 9) for both Qwen2.5-VL models tested, confirming the posterior is instruction-conditioned at inference time, an $n=200$, single-instruction-pass complement to the paired $n=150$ instruction-switch counterfactual (Finding 3), which is run on all four models (Table 5). We have not yet run this specific control on the two Qwen3-VL models; given that Finding 3’s paired counterfactual already holds for Qwen3-VL, we expect, but have not verified, the same qualitative collapse here.

Layer	14	15	16	17	18	19	20	21	22	23	24	25	26	27
Raw NLL	8.63	9.23	9.02	8.03	7.98	7.39	7.07	6.87	7.13	7.04	6.98	4.75	2.79	3.27
Tuned NLL	3.80	3.73	3.65	3.76	3.69	3.94	3.92	3.66	3.55	3.52	3.40	3.29	3.14	3.25

Table 8: **Tuned-lens control (GUI-Owl-7B)**. Per-layer coordinate-token NLL: raw logit lens vs. calibrated tuned lens (held-out). Both plateau at L26 ($L^\dagger$, bold). The tuned lens flattens the raw depth trend, attributing part of the raw decrease to an alignment confound, while preserving the plateau position.

Model	hit@1 real	hit@1 rand	GT-mass real	GT-mass rand
GUI-Owl-7B	40.5	1.5	0.292	0.010
UI-TARS-1.5-7B	38.0	1.5	0.253	0.009

Table 9: **Random-instruction control (ScreenSpot-Pro, $n=200$)**. Replacing the instruction with an unrelated sample’s collapses hit@1 to $\sim 1.5\%$ (near chance) and target-region mass to ~ 0.01 , ruling out a learned position prior.

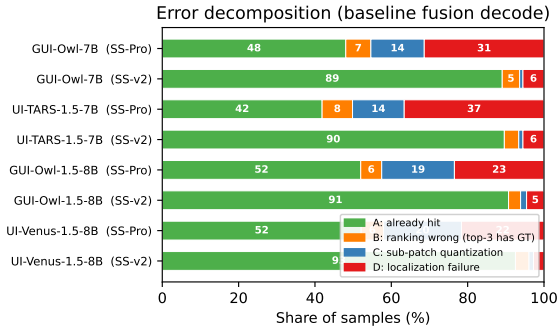


Figure 6: **Four-quadrant error decomposition of the fusion decode (referenced from the main paper’s retrofit-headroom analysis)**. **A**: already a strict hit; **B**: a top- k candidate is correct but ranking is wrong (Level 2 consensus targets this); **C**: a candidate patch overlaps the GT box but no point falls inside it, pure sub-patch quantization (the confidence gate targets this by deferring to the native decoder); **D**: no top- k candidate reaches the GT box (a localization failure no decoder can fix).

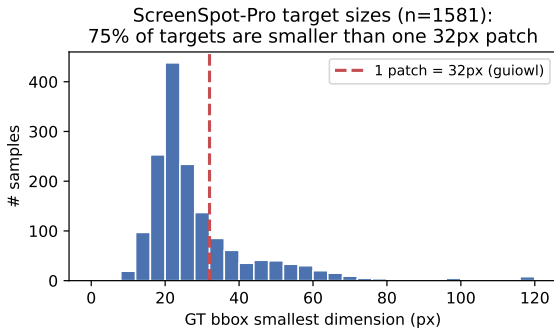


Figure 7: **Three quarters of ScreenSpot-Pro targets are smaller than one visual patch (referenced from §5.3)**. Distribution of the smaller GT-bbox dimension over all 1,581 ScreenSpot-Pro targets; the dashed line marks one patch width.

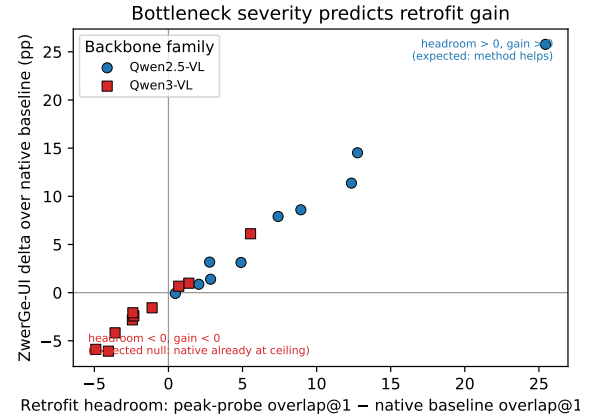


Figure 8: **Retrofit headroom predicts the ungated fusion-only gain (referenced from §5.3)**. x -axis: headroom $h = \max_{\ell} \text{ov}@1(p_{\ell}) - \text{ov}@1(\text{native})$; y -axis: strict, ungated fusion-only overlap@1 delta. Qwen2.5-VL backbones (positive headroom) sit in the positive-gain quadrant; Qwen3-VL backbones (near-zero headroom) sit near the origin.

Scope of controls and ongoing work. The controls above address sampling variance, the logit-lens alignment conundrum, dataset-level conflation of target size with resolution, and the instruction-prior alternative explanation directly. Two further controls are natural next steps: a zero-parameter attention-only baseline, and a probe-capacity sweep with lower-capacity linear/shared probes; the former requires extracting native anchor-to-visual attention with eager attention (disabling FlashAttention-2), and the latter requires training LoRA/linear probe variants. Our norm census (Fig. 5) already shows the backbone’s own pre-scale activations carry the depth-growing magnitude signal the probe reads, which is suggestive evidence that the inverted-U is not purely a probe-learning artifact.

C Statistical Significance of the Main-Table Gains

The main-table gains (Table 2) are small in magnitude. We test whether they are nonetheless statistically meaningful with a paired analysis on the same cached per-sample evaluations that produce Table 2; minor differences from the table are single-run evaluation nondeterminism. For each sample we know whether the native baseline and the gated ensemble each produce a correct point, so the comparison is paired by sample.

Discordant counts and McNemar’s test. Let b count samples the native decoder gets right but the ensemble gets wrong, and c count samples the ensemble gets right but the native gets wrong. The ensemble improves grounding iff $c > b$. McNemar’s test (with continuity correction) evaluates whether c exceeds b beyond chance. Table 10 reports b , c , the two-sided p -value, and a $3000\times$ bootstrap 95% confidence interval on the per-sample accuracy difference, for every cell.

Summary. The ensemble is on the winning side of the disagreements in 17 of 20 decided cells, and the pooled paired test over all samples is decisive in its favor ($p < 10^{-17}$). The improvement is individually significant ($p < 0.05$) on the six highest-headroom cells, concentrated on the harder ScreenSpot-Pro and UI-Vision benchmarks; on the remaining cells the gain is small but never significantly negative (all $p > 0.1$ when $c \leq b$). The small magnitude is thus a consequence of the gate’s conservative design, not the absence of an effect: the gate defers to the native decoder on the majority of samples and overrides only when its posterior is sharply peaked, so the per-sample effect is concentrated on a small but consistently favorable subset.

D Limitations

ZWERGE-UI operates at patch granularity, which introduces a localization ceiling on very high-resolution benchmarks (e.g., ScreenSpot-Pro) where targets may be smaller than a single patch. The patch granularity is a fundamental trade-off of the read-out approach: it avoids modifying the backbone vocabulary or training the language model head, but cannot achieve sub-patch precision without a second-stage

zoom step. Additionally, our ablations cover only the LoRA-probe architecture; the CrossAttn probe configuration space (number of heads, active layers, rank) was not exhaustively searched.

E Training and Evaluation Data

Training data. The probe and fusion heads are trained on 200,000 (image, instruction, bounding-box) grounding samples, released as `grounding_200k.jsonl`. Each record stores the screenshot path, the natural-language instruction, and the bbox in absolute pixel, normalized $[0, 1]$, and $[0, 1000]$ coordinates; a `source` field records the origin corpus. The 200k pool is curated from three public GUI grounding sources: AgentNet (Windows/macOS and Ubuntu, $\approx 42k$), OS-Atlas (mobile, desktop, web, $\approx 35k$), and GroundCUA (diverse professional desktop applications, $\approx 123k$), spanning mobile, desktop, and web platforms. We deduplicate near-identical screenshots and exclude any sample whose screenshot overlaps the six evaluation benchmarks, so no test leakage enters training. Both Stage 1 and Stage 2 train for one epoch on this pool with the backbone frozen (§2, §4.1).

Evaluation data. We evaluate on six benchmark configurations. **ScreenSpot-Pro** ($n=1581$), **ScreenSpot-v2** ($n=1271$), **OSWorld-G** (original split, $n=510$), **OSWorld-G-refine** (refusal-removed and reclassified split, $n=510$), **MMBench-GUI** ($n=3594$), and **UI-Vision** ($n=5790$). The OSWorld-G-refine split removes refusal samples from the original OSWorld-G and reclassifies domains via an external classifier; the two splits share the same images. Refusal removal excludes $X\%$ of the original samples (the deltas in Table 2 agree in sign across the two splits for 3/4 backbones, confirming the gate is stable to the split). Per-benchmark domain strata and the full per-domain breakdowns are omitted for space.

F Training-Free Posterior-to-Point Decoder

This section details the parameter-free decoder that produces the fusion point $\mathbf{f}$ from p_{final} , used by the confidence gate in §4.2. It has three composable stages.

Level 1: region re-ranking. Thresholding p_{final} at $0.3 \max$ with 4-connectivity BFS yields candidate regions $\{C_j\}$. We score each by total posterior mass normalised by $\sqrt{|C_j|}$, $S_j = \sum_{i \in C_j} p_{\text{final}}(i) / \sqrt{|C_j|}$, a compromise between mass (which favours large regions) and mean (which favours isolated spikes).

Level 2: cross-layer consensus. The same forward pass exposes $\{p_\ell\}$ and fusion weights ω_ℓ . For each dilated candidate $\tilde{C}_j$, we compute the per-layer region mass $m_{\ell j} = \sum_{i \in \tilde{C}_j} p_\ell(i)$ and the ω -weighted geometric mean $S_j^{\text{cons}} = \sum_\ell \omega_\ell \log(m_{\ell j} + \epsilon)$, which penalises peaks supported by only one layer.

Level 3: local Gaussian inversion. For the winning region we recover a sub-patch mode by fitting a weighted

Backbone	Benchmark	n	discordants		McNemar p	bootstrap 95% CI (pp)	
			b	c		lo	hi
GUI-Owl-7B	SS-Pro	1581	4	20	0.002	0.38	1.64
	SS-v2	1271	0	3	0.248	0.00	0.55
	OSW-G	510	5	9	0.423	-0.59	2.16
	OSW-G-orig	510	9	11	0.823	-1.37	1.96
	MMBench-GUI	3594	8	22	0.018	0.11	0.70
	UI-Vision	5790	47	76	0.012	0.12	0.86
UI-TARS-1.5-7B	SS-Pro	1581	3	25	< 0.001	0.76	2.09
	SS-v2	1271	0	4	0.134	0.08	0.71
	OSW-G	510	4	2	0.683	-1.37	0.59
	OSW-G-orig	510	5	7	0.773	-0.98	1.76
	MMBench-GUI	3594	9	19	0.089	0.00	0.58
	UI-Vision	5790	24	105	< 0.001	1.04	1.80
GUI-Owl-1.5-8B	SS-Pro	1581	8	10	0.814	-0.38	0.70
	SS-v2	1271	2	2	0.617	-0.31	0.31
	OSW-G	510	4	7	0.546	-0.59	1.96
	OSW-G-orig	510	8	5	0.579	-1.96	0.78
	MMBench-GUI	3594	13	25	0.074	0.00	0.67
	UI-Vision	5790	39	64	0.018	0.09	0.76
UI-Venus-1.5-8B	SS-Pro	1581	1	6	0.131	0.00	0.63
	SS-v2	1271	0	0	1.000	0.00	0.00
	OSW-G	510	2	2	0.617	-0.78	0.78
	OSW-G-orig	510	3	3	0.683	-0.98	0.98
	MMBench-GUI	3594	7	3	0.343	-0.28	0.06
	UI-Vision	5790	28	31	0.795	-0.21	0.31

Table 10: **Per-cell paired significance.** b : native correct, ensemble wrong; c : ensemble correct, native wrong; the ensemble improves grounding when $c > b$. Bold p are individually significant at 0.05. The pooled test over all samples is $b = 233$, $c = 461$ ($\chi^2=74.2$, $p<10^{-17}$); 17 of 20 decided cells have $c > b$ (binomial $p=0.001$); no cell is significantly worse than the baseline.

quadratic to $z = \log(p_{\text{final}} + \epsilon)$ on a 5×5 neighbourhood of the peak and solving $\Delta^* = -H^{-1}g$, inverting the anisotropic Gaussian geometry the probes were trained to emit. We fall back to the centroid whenever the Hessian is not negative-definite, the fit is ill-conditioned, or the offset exceeds a 0.75-cell clamp, bounding the damage rate on already-correct samples; an optional area gate ($|C| \leq 2$) is available as an ablation but is not applied in the main results. The target-size distribution that motivates operating at patch granularity in the first place is discussed in §5.3.

G Bottleneck Severity: Additional Detail

The main paper defines the retrofit headroom $h = \max_{\ell} \text{ov}@1(p_{\ell}) - \text{ov}@1(\text{native})$ and shows that it predicts the sign of the ungated fusion-only delta. We add one clarification here: the headroom diagnostic is computed *per (backbone, benchmark) cell* using the same A8 checkpoint-2800 probes and the same cached native predictions as the main results table, so it requires no additional inference beyond what the main results already use. Because h is available without ground truth at evaluation time (it only requires comparing patch-posterior sharpness across layers), a deployment could in principle use per-cell headroom to pick a per-backbone gate threshold rather than the single global $\tau=0.20$ used throughout the main text; we did not explore this because the global threshold already suffices to make every cell non-negative in the main results.

H Ablations

We ablate three design choices on UI-TARS-1.5-7B using a fast LoRA-probe variant (separate from the final CrossAttn probes), trained on 50k samples and evaluated at checkpoint 800. We vary GT label type (Binary vs. Gaussian), fusion scorer (Readiness-MLP vs. CosMeta), and probe layer range.

Table 11 shows three trends. First, fusion matters most on SS-Pro: replacing Readiness-MLP with CosMeta improves Binary GT (A0→A2) by +3.2pp. Second, GT label type interacts with fusion: Gaussian helps under Readiness (A0→A1) but lags Binary under CosMeta at this early checkpoint (A2 vs. A3). Third, layer range is critical: removing the deepest layers L26–L27 improves SS-Pro (A3→A4) by +3.5pp, consistent with late layers being serialization-oriented; truncating further to L18–L23 sharply hurts (A4→A5).

I Per-Layer Grounding Accuracy: Full Curves

Figure 9 shows the per-layer overlap@1 of every A7 CrossAttn probe evaluated independently (no fusion), plotted against the transformer layer index, for all four backbone models and all five benchmarks. Two training checkpoints are compared: **ckpt-800** (dashed blue, early training) and the **last checkpoint** (solid red, ckpt-3129 for Qwen2.5-VL models and ckpt-3130 for Qwen3-VL models). Horizontal dotted lines mark the corresponding fusion head accuracy.

Table 11: Ablation on UI-TARS-1.5-7B (overlap@1, %, checkpoint-800, fast LoRA-probe variant).

ID	GT	Fusion	Layers	SS-Pro	SS-v2	OSW-G
A0	Binary	Readiness	L18–27	46.4	89.3	72.0
A1	Gaussian	Readiness	L18–27	47.1	90.0	57.6
A2	Binary	CosMeta	L18–27	49.6	90.6	59.4
A3	Gaussian	CosMeta	L18–27	45.2	89.7	57.6
A4	Gaussian	CosMeta	L18–25	48.7	92.0	59.2
A5	Gaussian	CosMeta	L18–23	38.4	87.6	49.4

Setup. Each probe is a CrossAttn grounding head attached to a single transformer layer; all probes are trained simultaneously with independent per-layer KL losses (§2). At evaluation, each probe predicts independently – the fusion head (dotted line) combines all active probes and serves as the reference ceiling. Backbone parameters are completely frozen throughout.

Observations.

- **Inverted-U spatial profile.** All four models exhibit a consistent inverted-U shape: per-layer accuracy rises from the first probed layer, peaks around the middle of the probed range (L20–L22 for 28-layer models; L22–L25 for 36-layer models), then declines toward the final layers. This is direct evidence for the coordinate serialization bottleneck described in §3: the deepest layers progressively encode coordinate formatting rather than spatial content.
- **Training improves mid-layer probes more than shallow or deep ones.** Comparing ckpt-800 (blue) to the last checkpoint (red), the largest absolute gains appear in the middle of the probed range. Shallow probes improve less because spatial features at early layers are coarser; deep probes improve less because the layers are increasingly occupied by serialization signal that the probe cannot redirect.
- **Fusion tracks the (unknown) per-sample best layer.** The fusion dotted line stays within 1–3 pp of the peak of the per-layer curve on 16 of the 20 (model, benchmark) cells and exceeds it on the remaining 4; on ScreenSpot-Pro and ScreenSpot-v2 it falls 0.8–2.7 pp below the oracle peak. This is the intended behaviour: fusion is a single, sample-adaptive, deployable predictor that does not require per-benchmark layer selection, which the layerwise analysis (§3.1) shows to be non-monotonic and backbone-dependent.
- **Qwen3-VL probes cover a wider band.** GUI-Owl-1.5-8B and UI-Venus-1.5-8B (36-layer Qwen3-VL, probes at L18–L35) show a broader and flatter peak compared to the 28-layer models (probes at L14–L27). The wider

effective range suggests that the spatial-to-serialization transition is more gradual in deeper networks.

Figures 10–14 show the same data broken out by benchmark, at a larger scale for readability.

A7 CrossAttn Probe: Per-Layer overlap@1 (dashed=ckpt-800, solid=last ckpt)
Dotted horizontal lines show the fusion head accuracy

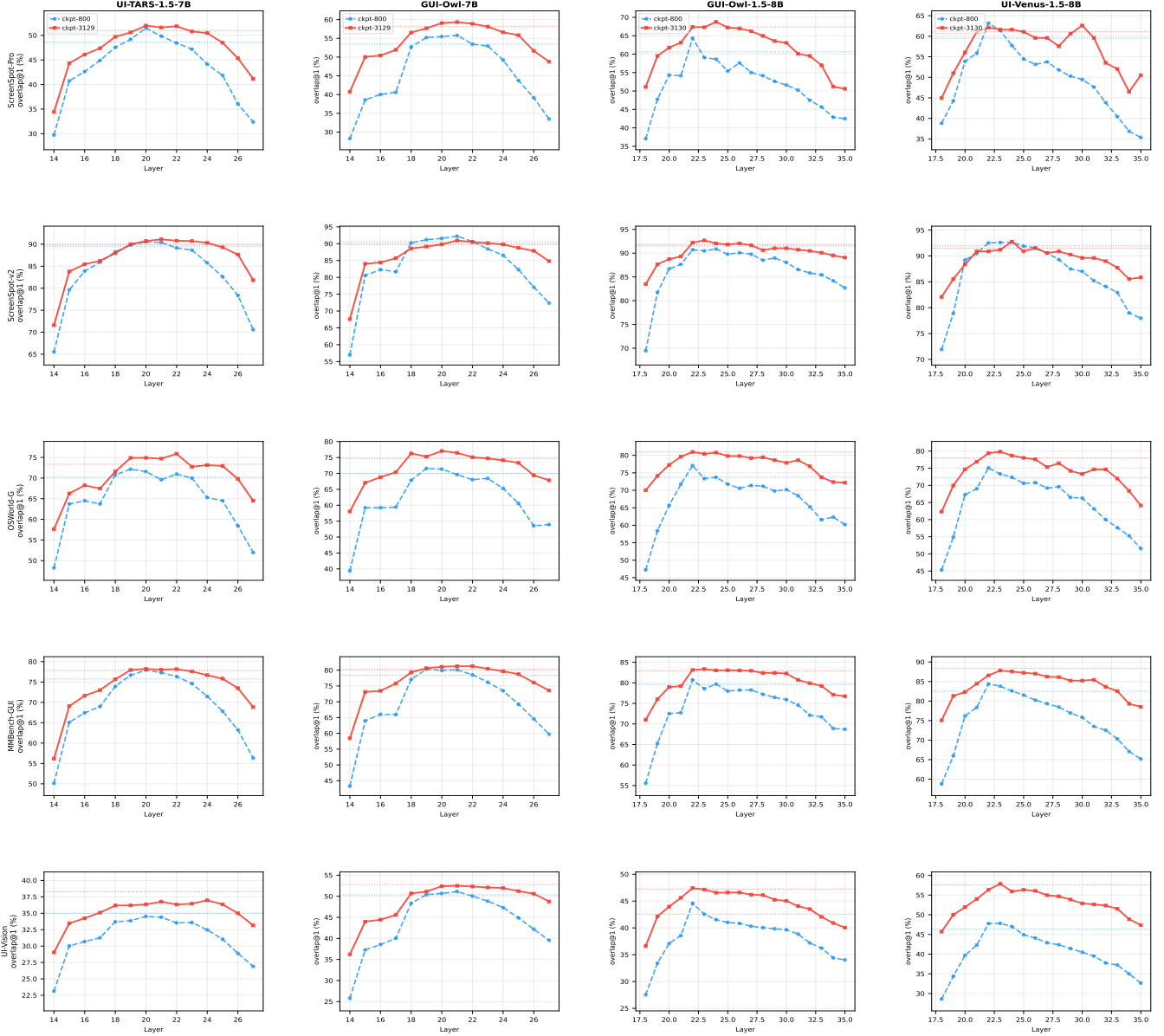


Figure 9: Per-layer overlap@1 of A7 CrossAttn probes across all four backbone models and five benchmarks. Each cell shows overlap@1 for a single probe layer evaluated independently (no cross-layer fusion). **Rows:** ScreenSpot-Pro (SS-Pro), ScreenSpot-v2 (SS-v2), OSWorld-G, MMBench-GUI, UI-Vision. **Columns:** UI-TARS-1.5-7B, GUI-Owl-7B, GUI-Owl-1.5-8B, UI-Venus-1.5-8B. **Blue dashed:** ckpt-800 (early training); **red solid:** last checkpoint (ckpt-3129 / ckpt-3130). **Horizontal dotted lines:** corresponding fusion head accuracy (the final ZWERGE-UI output). All four models exhibit an inverted-U spatial profile: accuracy peaks at intermediate layers and declines at the deepest layers, consistent with the coordinate serialization bottleneck (§3). The fusion head (dotted) tracks the best single-layer probe within 1–3 pp across nearly all panels (and exceeds it on a few), providing a deployable, sample-adaptive readout that does not require per-benchmark layer selection.

Per-Layer overlap@1 on ScreenSpot-Pro
(A7 CrossAttn Probe, each probe layer evaluated independently)

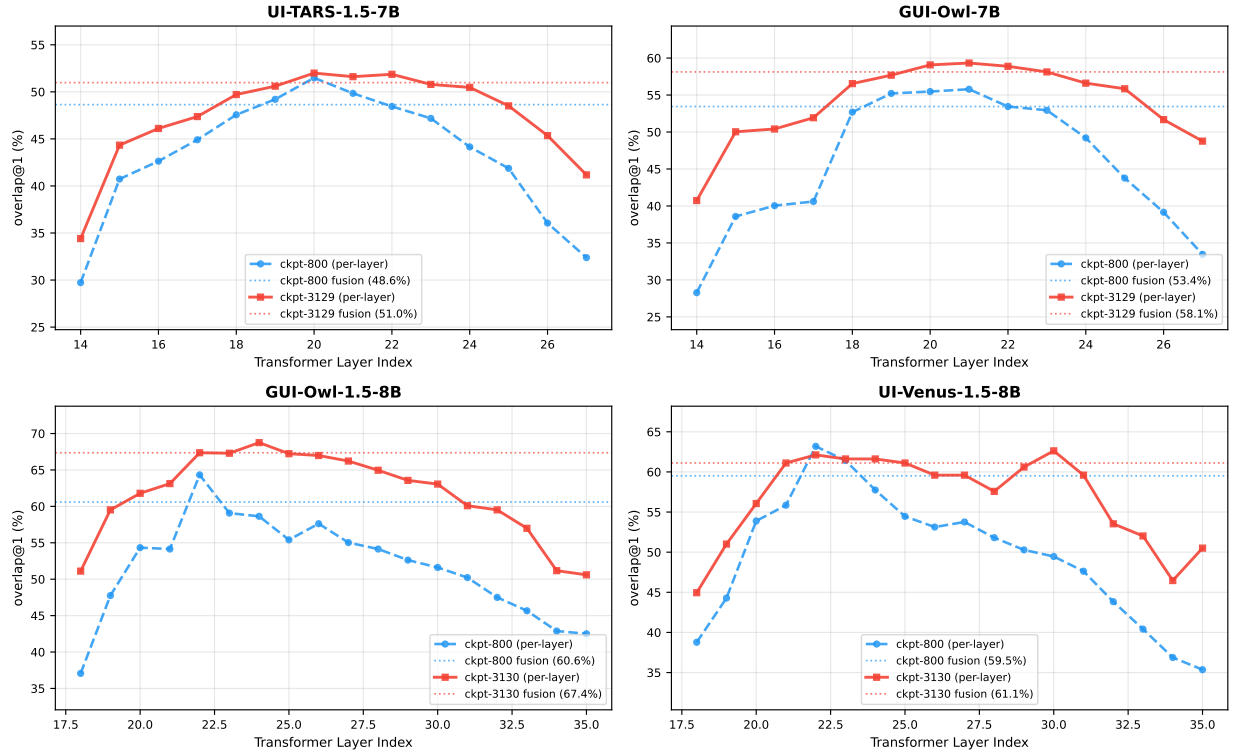


Figure 10: **Per-layer overlap@1 on ScreenSpot-Pro.** ScreenSpot-Pro is the most demanding benchmark (fine-grained targets on high-resolution desktop screenshots). The inverted-U profile is the sharpest here, with the deepest layers showing the steepest decline, reflecting that high-resolution targets require the most precise spatial encoding and are most sensitive to the serialization shift in deep layers. Note that the vertical scales differ across panels to emphasize the shape of the curve within each model.

Per-Layer overlap@1 on ScreenSpot-v2
(A7 CrossAttn Probe, each probe layer evaluated independently)

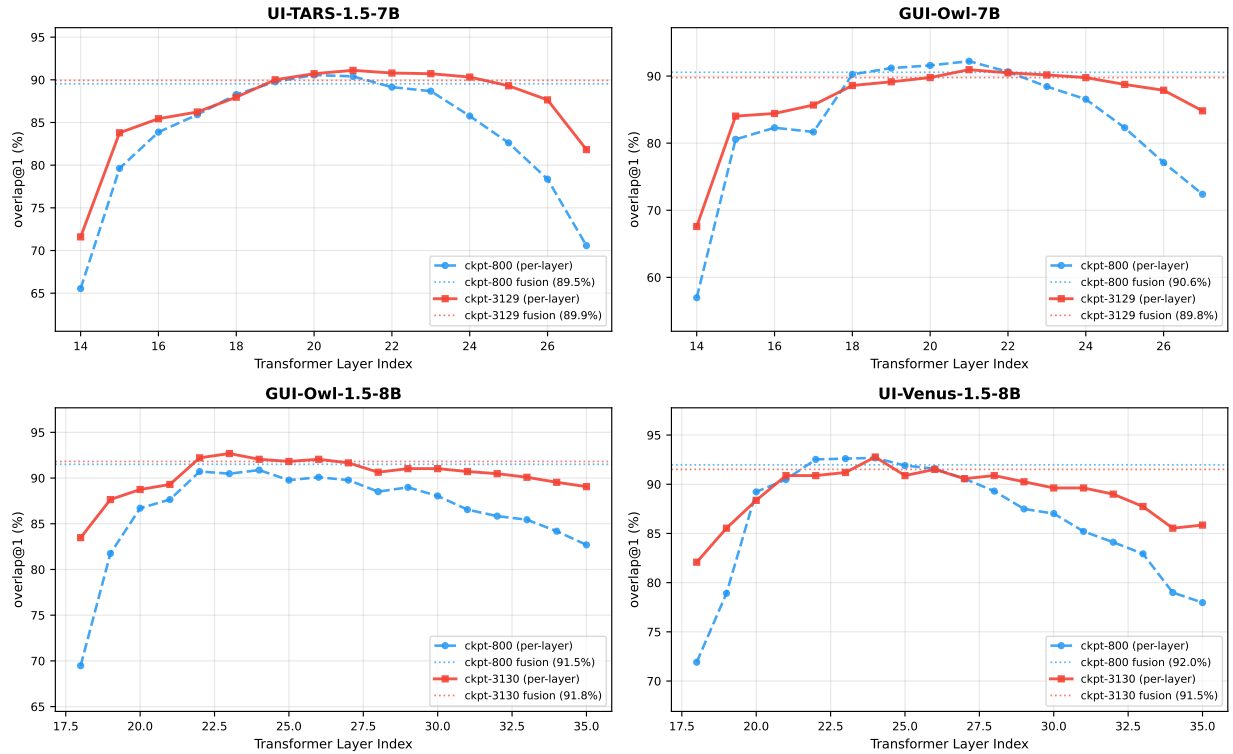


Figure 11: **Per-layer overlap@1 on ScreenSpot-v2.** ScreenSpot-v2 is a moderate-resolution benchmark (mobile, desktop, web) where overall accuracy is already high (>87% at ckpt-800). The inverted-U is shallower and the peak is broader compared to ScreenSpot-Pro, indicating that moderate-resolution grounding is less sensitive to which specific layer is read.

Per-Layer overlap@1 on OSWorld-G
(A7 CrossAttn Probe, each probe layer evaluated independently)

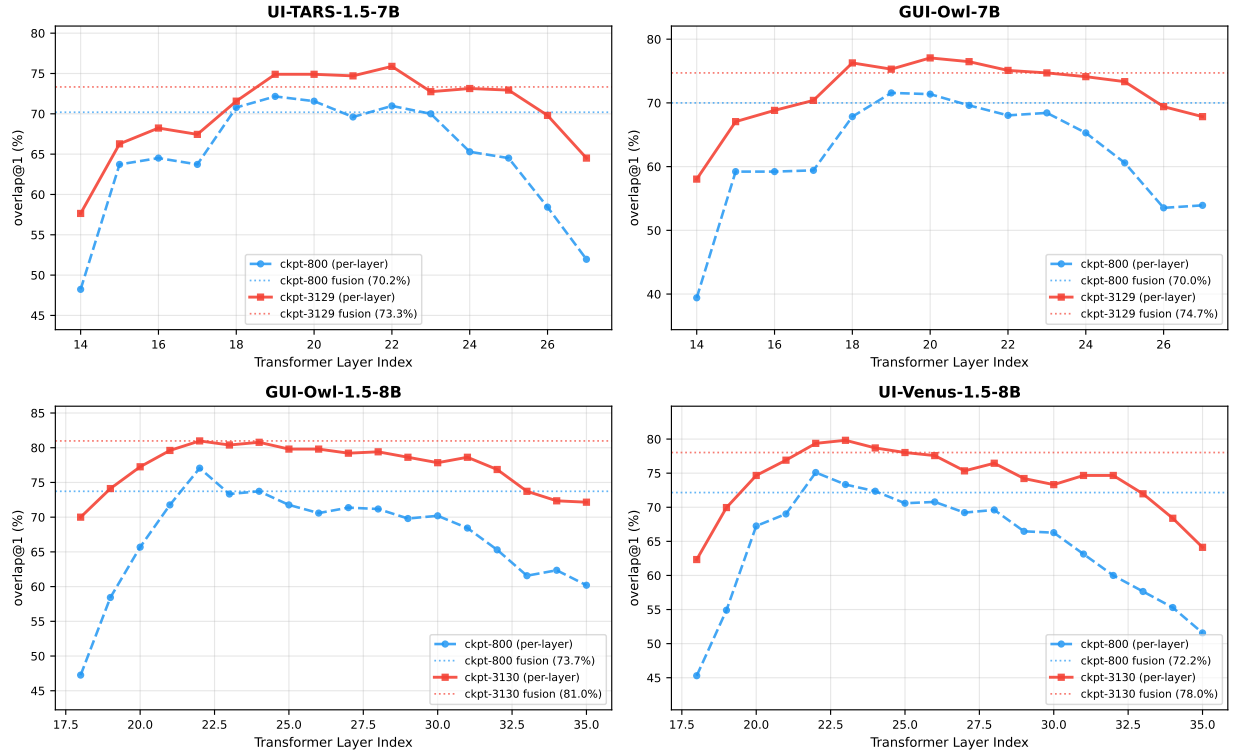


Figure 12: **Per-layer overlap@1 on OSWorld-G**. OSWorld-G provides real computer-task GUI samples with diverse task types (Text Matching, Element Recognition, Layout Understanding, Fine-grained Manipulation). The per-layer profile shows a similar inverted-U shape but with a plateau over a wider middle range ($\sim$ L18–L24 for 28-layer models), suggesting that layout-level spatial reasoning is distributed more broadly across intermediate layers than fine-grained element targeting.

Per-Layer overlap@1 on MMBench-GUI
(A7 CrossAttn Probe, each probe layer evaluated independently)

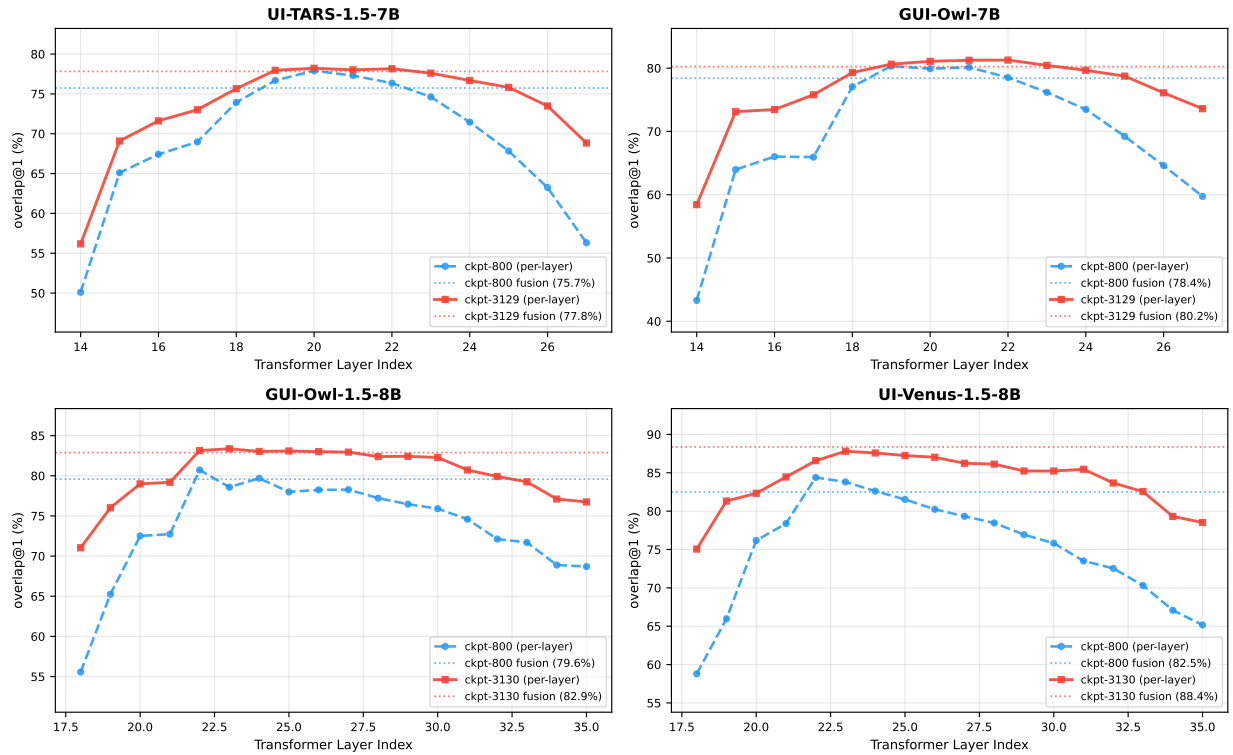


Figure 13: **Per-layer overlap@1 on MMBench-GUI.** MMBench-GUI covers six platforms (Windows, macOS, Linux, iOS, Android, Web). The per-layer curves are notably high even at ckpt-800 (blue dashed) for Qwen3-VL models (GUI-Owl-1.5-8B and UI-Venus-1.5-8B), reflecting that these models have strong cross-platform generalization already encoded in their intermediate representations. The gap between ckpt-800 and last checkpoint is smaller on MMBench-GUI than on ScreenSpot-Pro, suggesting that the per-layer signal saturates earlier on this benchmark.

Per-Layer overlap@1 on UI-Vision
(A7 CrossAttn Probe, each probe layer evaluated independently)

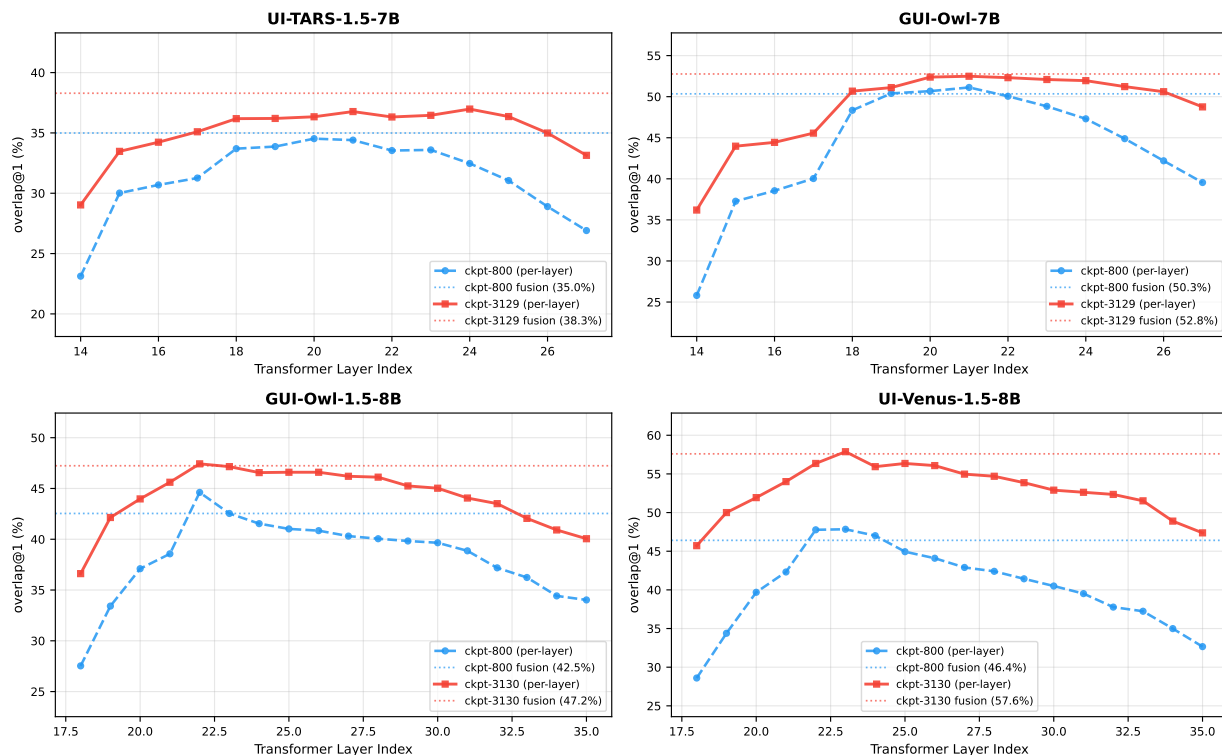


Figure 14: **Per-layer overlap@1 on UI-Vision.** UI-Vision decomposes grounding into Basic, Functional, Spatial, and Layout subtypes; accuracy here represents the weighted average over all 5,790 samples. Per-layer accuracy is substantially lower on UI-Vision than on other benchmarks, particularly for Qwen2.5-VL models at early training (ckpt-800 for UI-TARS-1.5-7B peaks below 30%), indicating that UI-Vision’s diverse task composition is harder for intermediate probes to solve with limited training. The inverted-U is still present but shallower and shifted earlier (peak around L18–L20), possibly because UI-Vision includes layout-level tasks that require higher-level semantic understanding.

Prompt for the UI-TARS-1.5 Model

```

1 You are a helpful assistant.
2
3 You are a GUI agent. You are given a task and your action history, with screenshots.
4 You need to perform the next action to complete the task.
5
6 ## Output Format
7 Action: ...
8
9 ## Action Space
10 click(start_box='<|box_start|>(x1,y1)<|box_end|>')
11
12 ## User Instruction
13 Task Instruction: {instruction}

```

Prompt for the GUI-Owl Model

```
1 You are a helpful assistant.
2 # Tools
3 You may call one or more functions to assist with the user query.
4 You are provided with function signatures within <tools></tools> XML tags:
5 <tools>
6 {
7   "type": "function",
8   "function": {
9     "name": "computer_use",
10    "description": "Use a mouse to interact with a computer screen."
11      "* The screen's resolution is ${height}x${width}."
12      "* Make sure to click any buttons, links, icons, etc with the cursor tip in the center of the
13        element. Don't click boxes on their edges unless asked.",
14    "parameters": {
15      "properties": {
16        "action": {
17          "description": "The action to perform. The available actions are:"
18            "* 'left_click': Click the left mouse button at a specified (x, y) pixel coordinate
19              on the screen.",
20          "enum": ["left_click"],
21          "type": "string"
22        },
23        "coordinate": {
24          "description": "(x, y): The x (pixels from the left edge) and y (pixels from the top edge)
25            coordinates to click.",
26          "type": "array"
27        }
28      },
29      "required": ["action"],
30      "type": "object"
31    }
32  }
33 </tools>
34 For each function call, return a json object with function name and arguments within <tool_call></tool_call>
35   XML tags:
36 <tool_call>
37 {"name": <function-name>, "arguments": <args-json-object>}
38 </tool_call>
39 ## User Instruction
40 Please complete the following tasks by clicking using 'left_click' function: {instruction}
```

Prompt for the GUI-Owl-1.5 Model

```
1  # Tools
2
3  You may call one or more functions to assist with the user query.
4
5  You are provided with function signatures within <tools></tools> XML tags:
6  <tools>
7  {"type": "function", "function": {"name": "computer_use", "description": "Use a mouse to interact with a
   computer.\n* The screen's resolution is 1000x1000.\n* Make sure to click any buttons, links, icons, etc
   with the cursor tip in the center of the element. Don't click boxes on their edges unless asked.", "
   parameters": {"properties": {"action": {"description": "The action to perform. The available actions are
   :\n* 'left_click': Click the left mouse button with coordinate (x, y) pixel coordinate on the screen.",
   "enum": ["left_click"], "type": "string"}, "coordinate": {"description": "(x, y): The x (pixels from the
   left edge) and y (pixels from the top edge) coordinates to click.", "type": "array"}}, "required": ["
   action"], "type": "object"}}}
8  </tools>
9
10 For each function call, return a json object with function name and arguments within <tool_call></tool_call>
    XML tags:
11 <tool_call>
12 {"name": <function-name>, "arguments": <args-json-object>}
13 </tool_call>
14
15 ## User Instruction
16 {instruction}
```

Prompt for the UI-Venus-1.5 Model

```
1  Output the center point of the position corresponding to the following instruction: {instruction}.
2
3  The output should just be the coordinates of a point, in the format [x,y].
```

Prompt for the UI-TARS-1.5 Retrofit Model

```
1  [System]
2  You are a GUI agent. You are given a task and a screenshot.
3  You need to perform the next action to complete the task.
4
5  ## Output Format
6
7  Action: ...
8
9  ## Action Space
10
11 click(start_box='<|ground|><|pointer_start|><|pointer_pad|><|pointer_end|>')
12
13 [User]
14 <image>
15 {instruction}
16
17 [Assistant]
18 click(start_box='<|ground|><|pointer_start|><|pointer_pad|><|pointer_end|>')
```

Prompt for the GUI-Owl-7B / GUI-Owl-1.5 Retrofit Model

```
1 [System]
2 # Tools
3
4 You may call one or more functions to assist with the user query.
5
6 You are provided with function signatures within <tools></tools> XML tags:
7 <tools>
8 {"type": "function", "function": {
9   "name": "computer_use",
10  "description": "Use a mouse and keyboard to interact with a computer, and take screenshots. This is an
      interface to a desktop GUI. You do not have access to a terminal or applications menu. You must click on
      desktop icons to start applications. * The screen's resolution is 1000x1000.\n* Make sure to click
      buttons, links, icons, etc with the cursor tip in the center of the element. Don't click boxes on their
      edges unless asked.",
11  "parameters": {"properties": {"action": {"description": "The action to perform. The available actions are:\n*
      'left_click': Click the left mouse button at coordinate (x, y) pixel coordinate on the screen.", "enum
      ": ["left_click"], "type": "string"}, "coordinate": {"description": "(x, y): The x (pixels from the left
      edge) and y (pixels from the top edge) coordinates to click. Required only by 'action=left_click'.", "
      type": "array"}}, "required": ["action", "coordinate"], "type": "object"}}}
12 </tools>
13
14 For each function call, return a json object with function name and arguments within <tool_call></tool_call>
    XML tags:
15 <tool_call>
16 {"name": "computer_use", "arguments": {"action": "left_click", "coordinate": [<|ground|><|pointer_start|><|
    pointer_pad|><|pointer_end|>]}}
17 </tool_call>
18
19 [User]
20 <image>
21 {instruction}
22
23 [Assistant]
24 <tool_call>
25 {"name": "computer_use", "arguments": {"action": "left_click", "coordinate": [<|ground|><|pointer_start|><|
    pointer_pad|><|pointer_end|>]}}
26 </tool_call>
```

Prompt for the UI-Venus-1.5 Retrofit Model

```
1 [User]
2 <image>
3 Output the center point of the position corresponding to the following instruction:
4 {instruction}.
5
6 The output should just be the coordinates of a point, in the format
7 [<|ground|><|pointer_start|><|pointer_pad|><|pointer_end|>].
8
9 [Assistant]
10 [<|ground|><|pointer_start|><|pointer_pad|><|pointer_end|>]
```



Figure 15: Successful cases of UI-TARS-1.5-7B on ScreenSpot-Pro

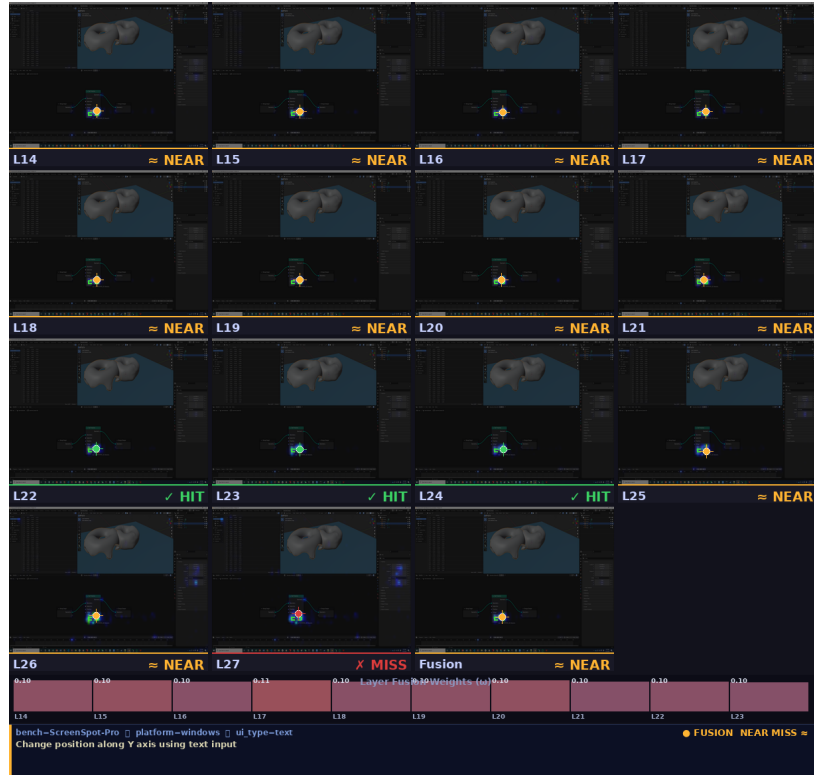


Figure 16: Failure cases of UI-TARS-1.5-7B on ScreenSpot-Pro



Figure 17: Successful cases of UI-Venus-1.5-8B on MMBench-GUI-L2

